

Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Sistema de Base de Datos 1
Ing. Bryan Gómez



Tema:
PROYECTO 1
BASE DE DATOS HOSPITALARIA
HOSPITALES DE OCCIDENTE

Selvyn Estuardo Ixtabalan Tistoj 201930524

Quetzaltenango, 28 de Septiembre de 2026

Contenido

1. Alcance y diseño	3
2. Modelo de datos	3
3. Diccionario de datos	12
Organización.....	12
Personas y personal	17
Expediente clínico.....	22
Consulta y emergencia	27
Cirugía.....	35
Hospitalización	43
Facturación y calidad	45
4. Sentencias SQL	49
5. Comprobación y resultados	109
6. Conclusiones	109

1. Alcance y diseño

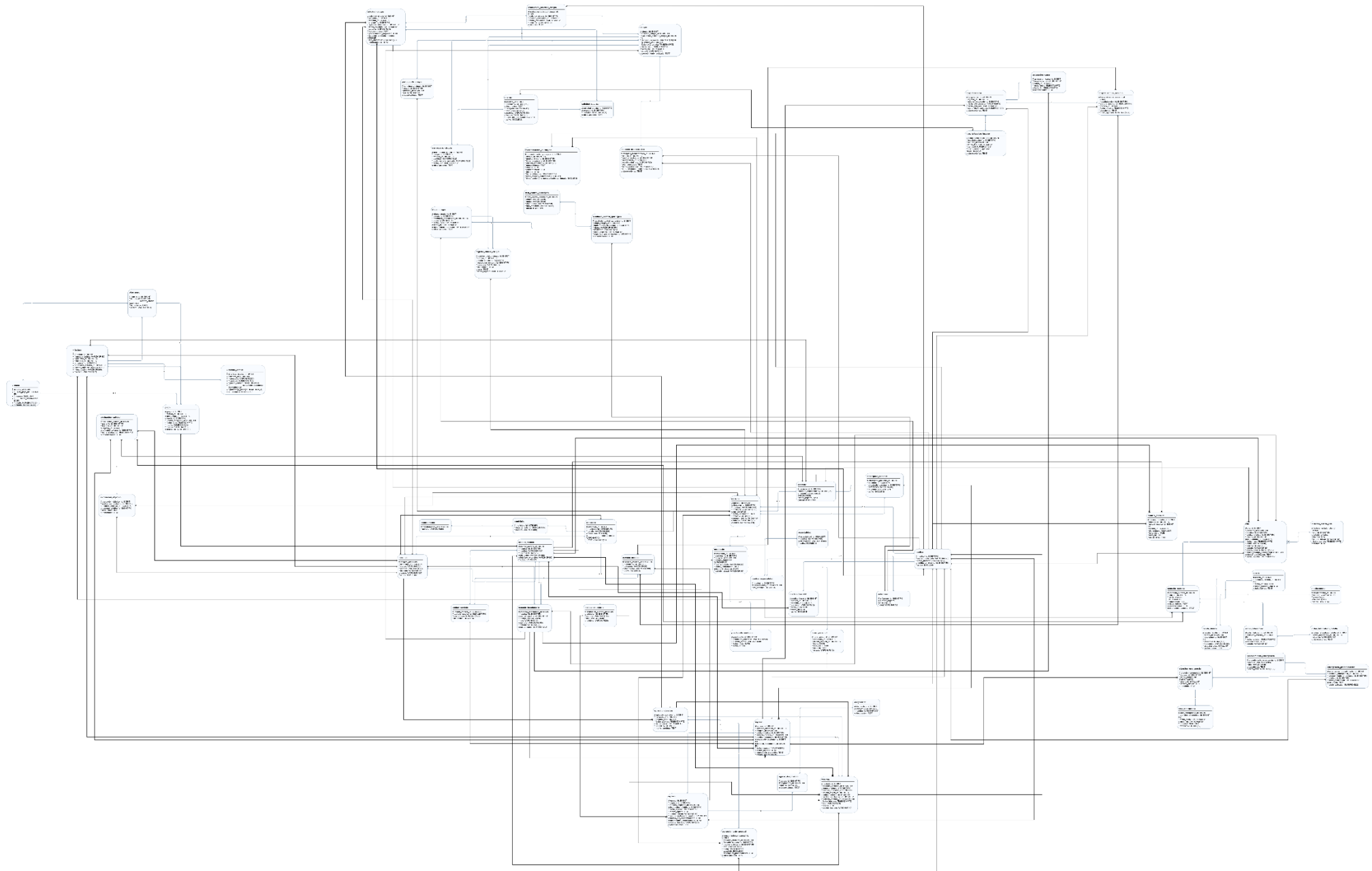
La base almacena hospitales, personas, atención en las cuatro unidades médicas, movimientos clínicos, facturación y calificaciones. El modelo se implementó en PostgreSQL 16 con 64 tablas y 124 llaves foráneas. Los datos de demostración son completamente ficticios.

Las direcciones y las personas se registran una vez y se relacionan con pacientes y profesionales mediante claves. Los precios aplicados se guardan en los hechos históricos, aunque cambie el precio del catálogo. Las relaciones muchos a muchos se resuelven con tablas asociativas.

2. Modelo de datos

El archivo editable `modelo_hospitalario.drawio` incluye ocho páginas. En los atributos, # indica identificador, * obligatoriedad y o opcionalidad. FK señala una clave foránea. Las láminas siguientes permiten inspeccionar cada módulo por separado.

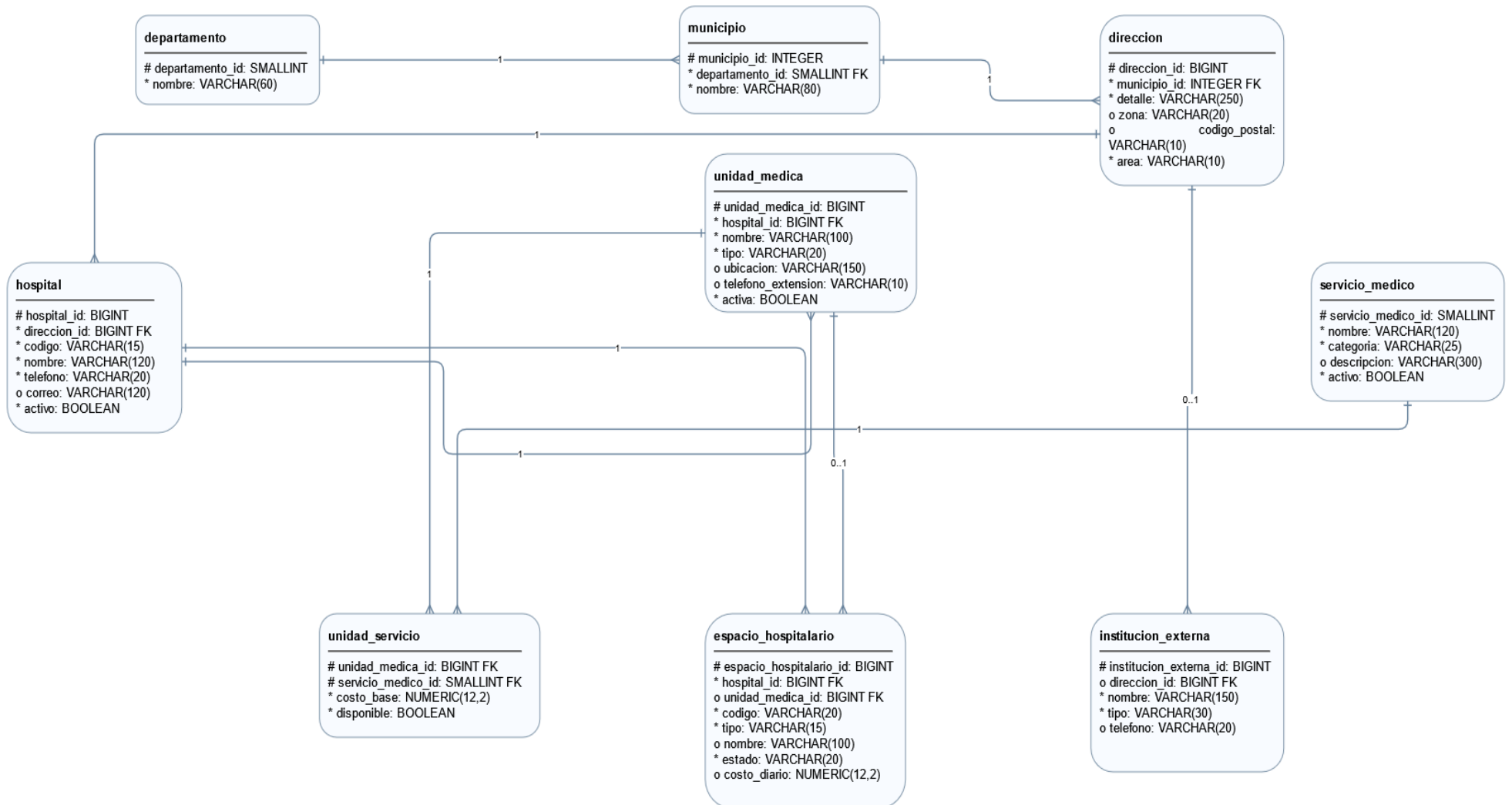
DIAGRAMA GENERAL:



2.1 Organización

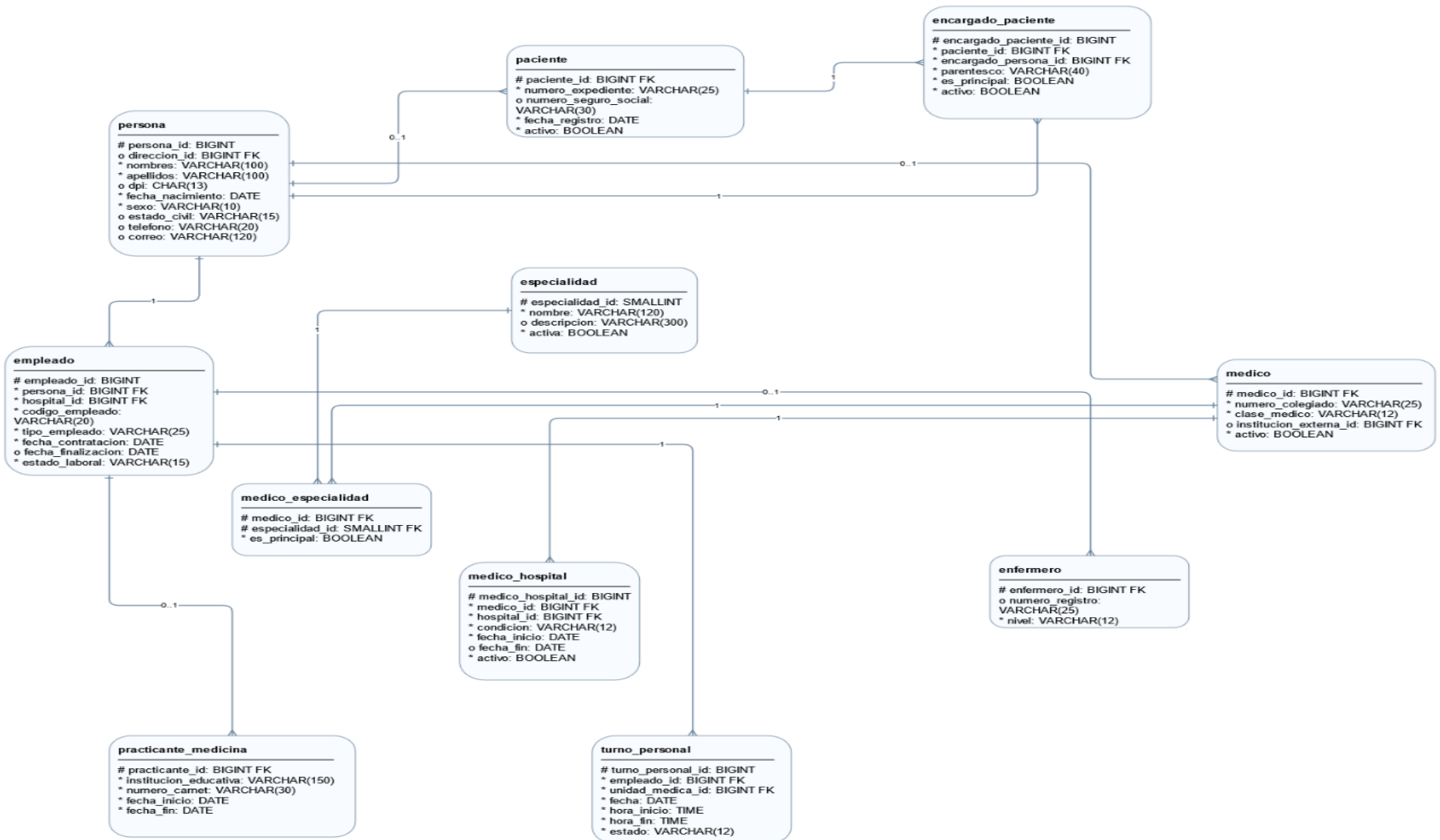
1. Organizacion

Barker: # identificador | * obligatorio | o opcional | FK clave foranea



2. Personas y personal

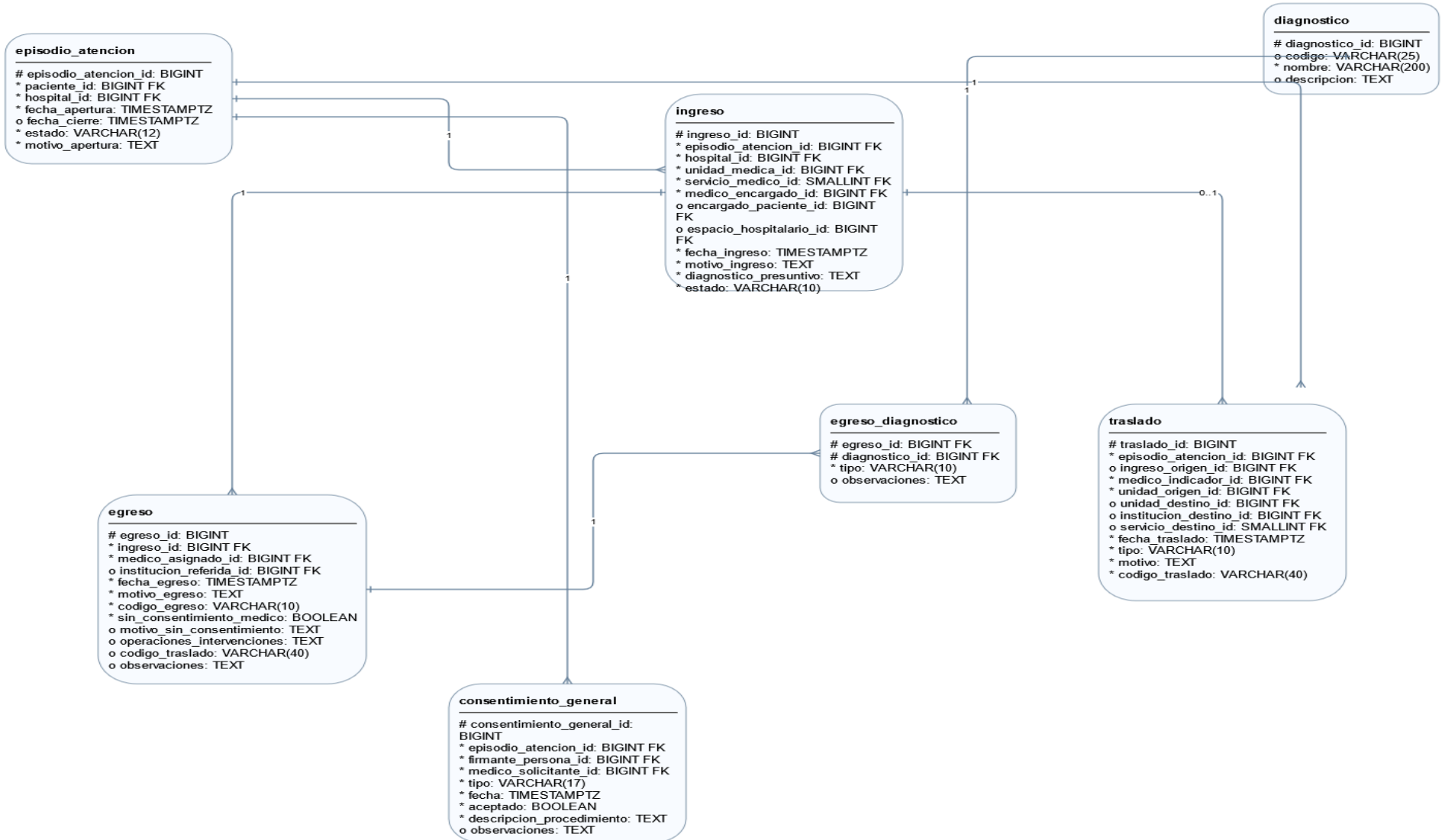
Barker: # identificador | * obligatorio | o opcional | FK clave foranea



2.3 Expediente clínico

3. Expediente clínico

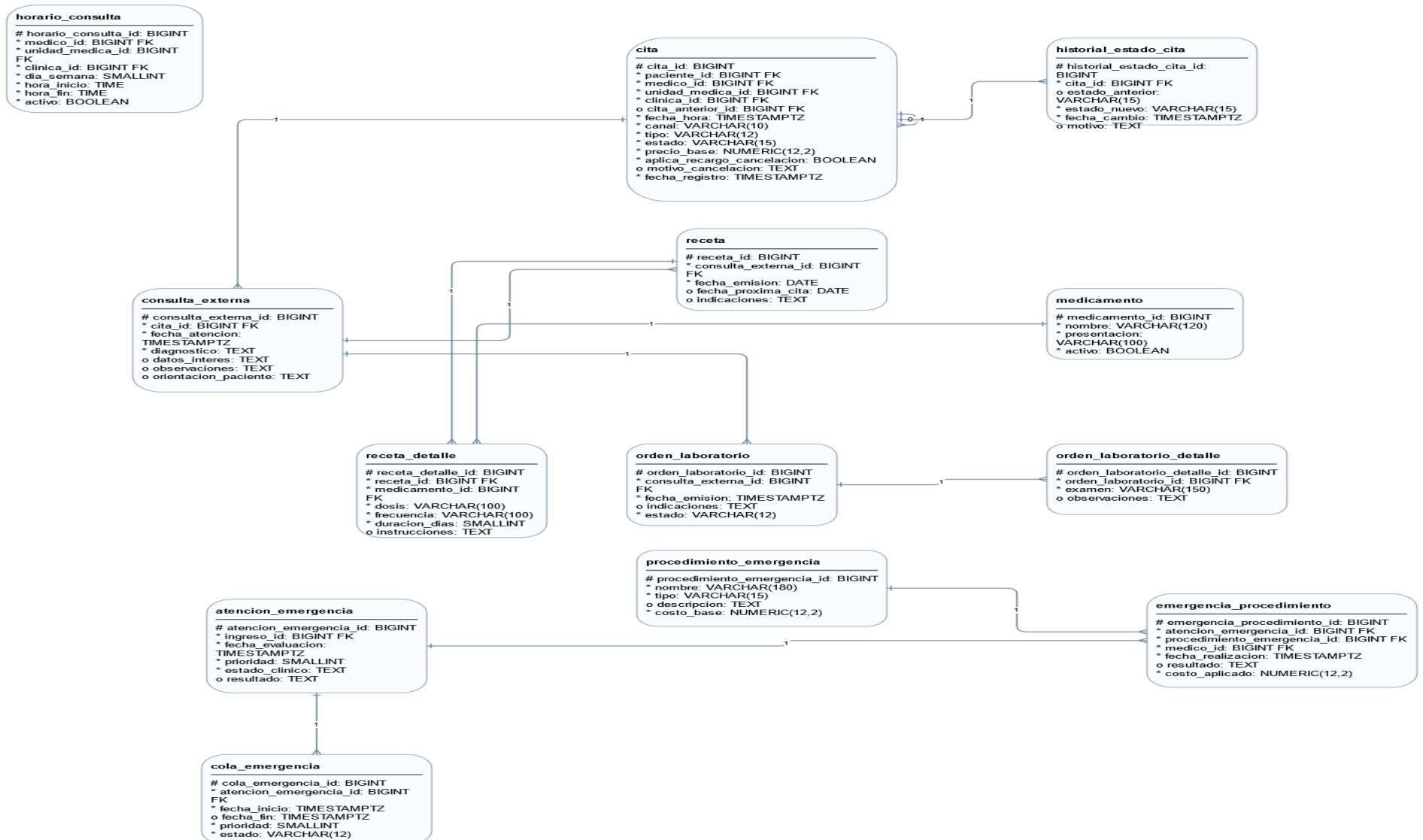
Barker: # identificador | * obligatorio | o opcional | FK clave foranea



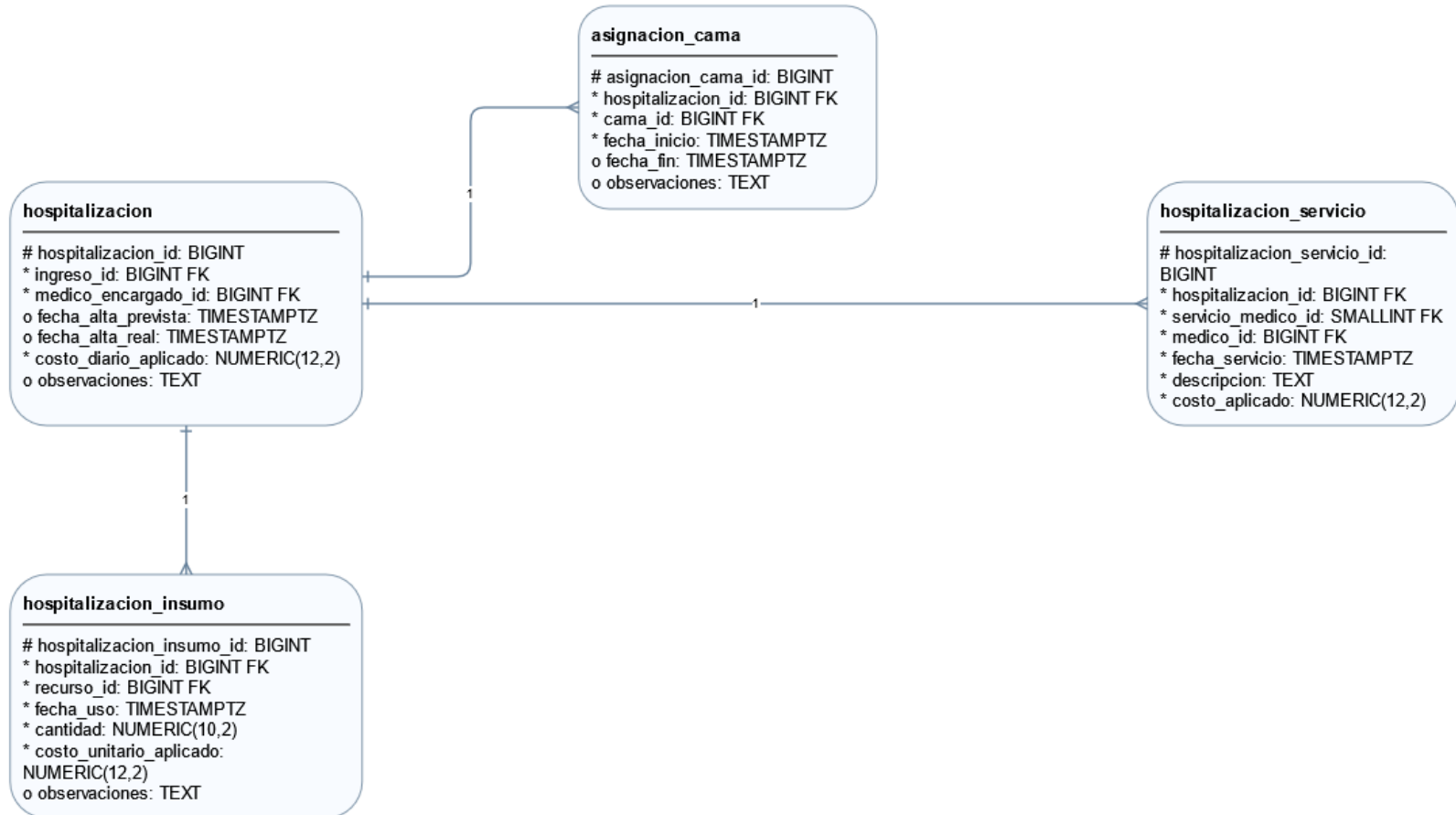
2.4 Consulta y emergencia

4. Consulta y emergencia

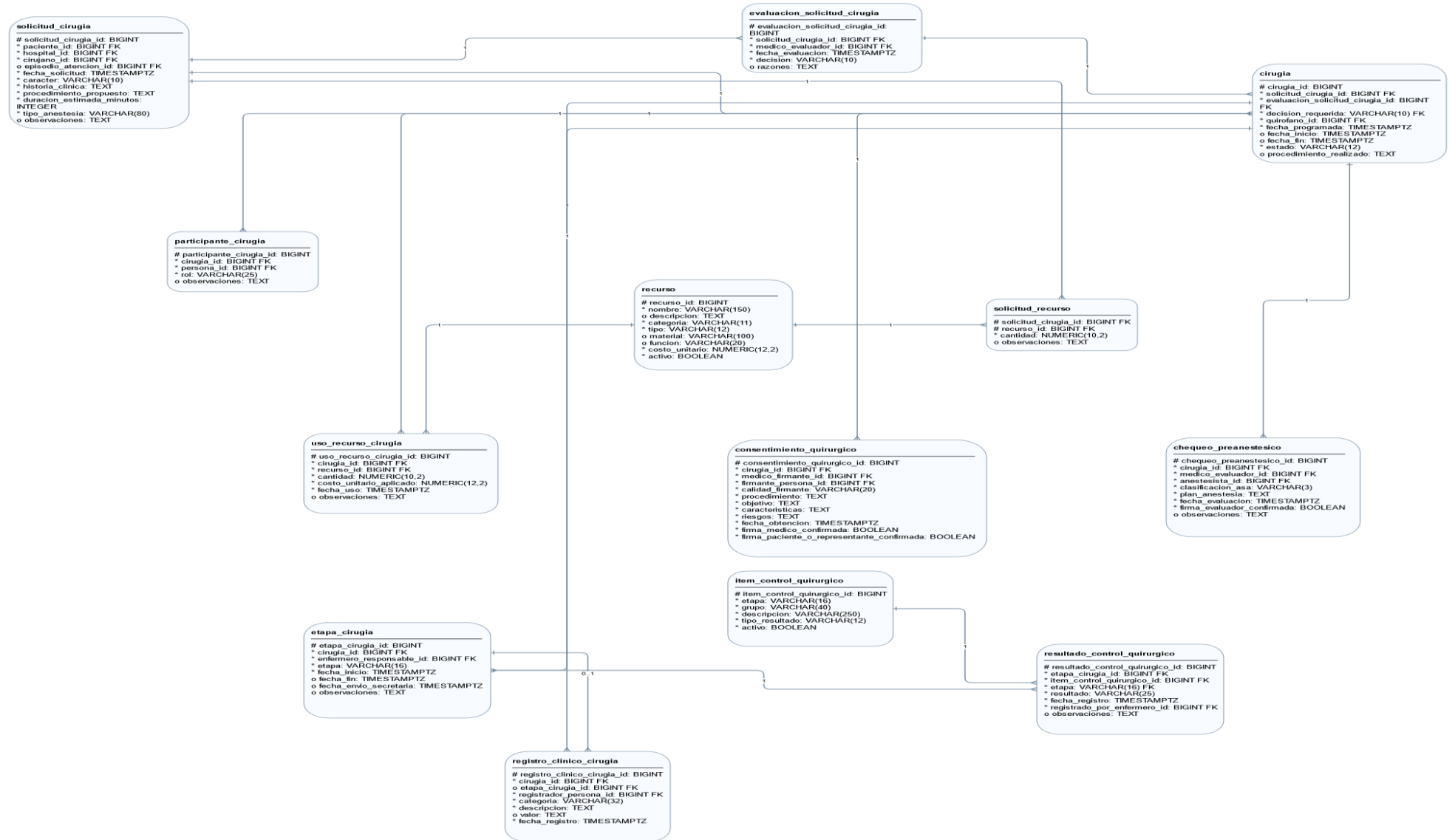
Barker: # identificador | * obligatorio | o opcional | FK clave foranea



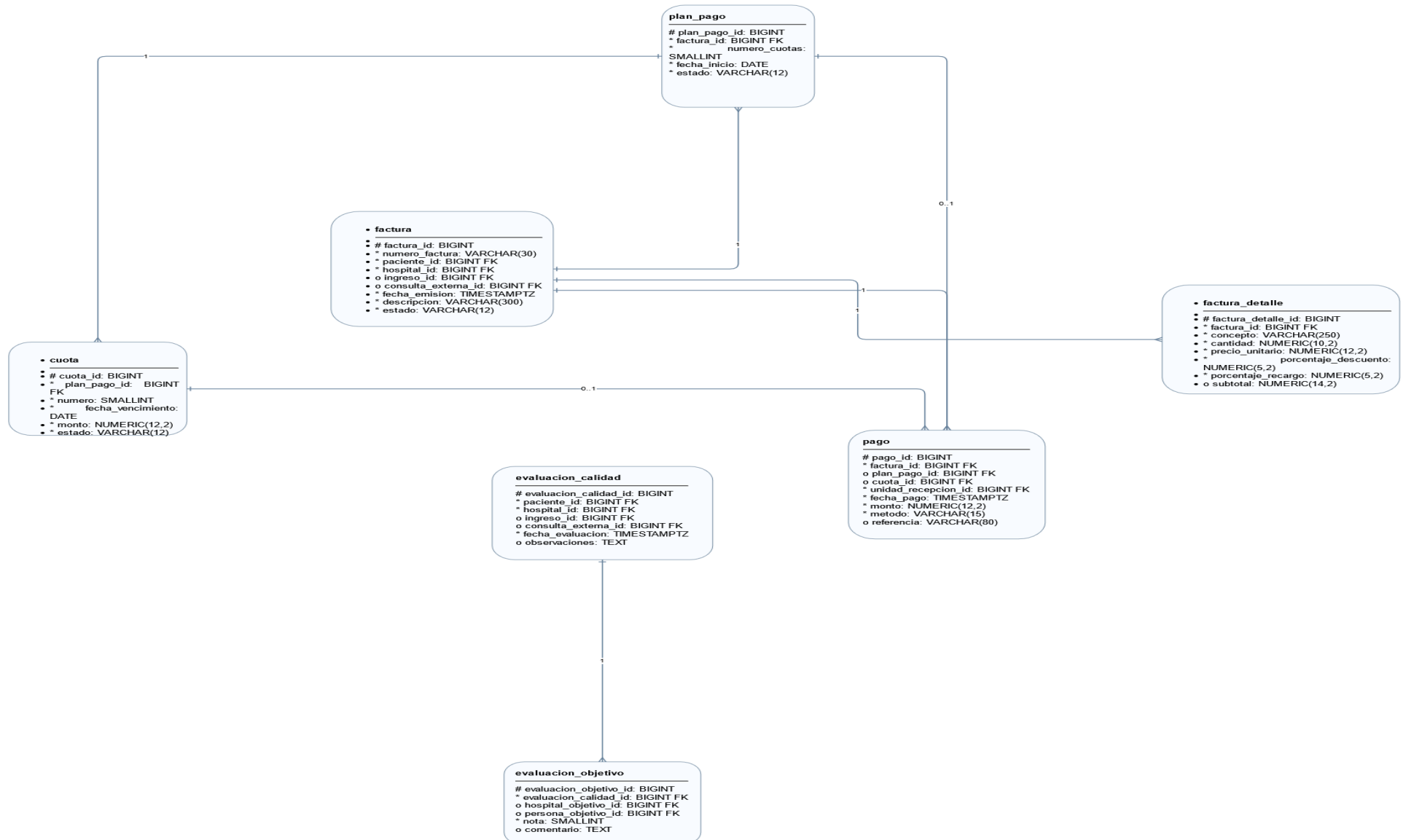
2.5 Hospitalización



2.6 Cirugía



2.7 Facturación y calidad



3. Diccionario de datos

Cada tabla indica tipo, llave, nulabilidad, función del atributo y las acciones de sus llaves foráneas. Las justificaciones de DELETE se basan en la conservación clínica y financiera o en la dependencia del detalle. UPDATE CASCADE propaga cambios de la clave referenciada.

Organización

Departamento

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
departamento_id	SMALLINT	PK	No	Identificador único del registro	—	—	—
nombre	VARCHAR(60)	—	No	Nombre	—	—	—

Municipio

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
municipio_id	INTEGER	PK	No	Identificador único del registro	—	—	—
departamento_id	SMALLINT	FK	No	Referencia a departamento	RESTRICT	CASCADE	Conserva el registro relacionado de departamento y su historial.
nombre	VARCHAR(80)	—	No	Nombre	—	—	—

Dirección

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
direccion_id	BIGINT	PK	No	Identificador único del registro	—	—	—
municipio_id	INTEGER	FK	No	Referencia a municipio	RESTRICT	CASCADE	Conserva el registro relacionado de municipio y su historial.
detalle	VARCHAR(250)	—	No	Detalle	—	—	—
zona	VARCHAR(20)	—	Sí	Zona	—	—	—
codigo_postal	VARCHAR(10)	—	Sí	Codigo postal	—	—	—
area	VARCHAR(10)	—	No	Area	—	—	—

Hospital

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
hospital_id	BIGINT	PK	No	Identificador único del registro	—	—	—
direccion_id	BIGINT	FK	No	Referencia a direccion	RESTRICT	CASCADE	Conserva el registro relacionado de direccion y su historial.
codigo	VARCHAR(15)	—	No	Codigo	—	—	—
nombre	VARCHAR(120)	—	No	Nombre	—	—	—
telefono	VARCHAR(20)	—	No	Telefono	—	—	—
correo	VARCHAR(120)	—	Sí	Correo	—	—	—
activo	BOOLEAN	—	No	Activo	—	—	—

unidad_medica

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
unidad_medica_id	BIGINT	PK	No	Identificador único del registro	—	—	—
hospital_id	BIGINT	FK	No	Referencia a hospital	RESTRICT	CASCADE	Conserva el registro relacionado de hospital y su historial.
nombre	VARCHAR(100)	—	No	Nombre	—	—	—
tipo	VARCHAR(20)	—	No	Tipo	—	—	—
ubicacion	VARCHAR(150)	—	Sí	Ubicacion	—	—	—
telefono_extension	VARCHAR(10)	—	Sí	Telefono extension	—	—	—
activa	BOOLEAN	—	No	Activa	—	—	—

servicio_medico

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
servicio_medico_id	SMALLINT	PK	No	Identificador único del registro	—	—	—
nombre	VARCHAR(120)	—	No	Nombre	—	—	—
categoria	VARCHAR(25)	—	No	Categoria	—	—	—

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
descripcion	VARCHAR(300)	—	Sí	Descripcion	—	—	—
activo	BOOLEAN	—	No	Activo	—	—	—

unidad_servicio

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
unidad_medica_id	BIGINT	PK/FK	No	Identificador único del registro	CASCADE	CASCADE	El detalle depende del registro de unidad_medica.
servicio_medico_id	SMALLINT	PK/FK	No	Identificador único del registro	RESTRICT	CASCADE	Conserva el registro relacionado de servicio_medico y su historial.
costo_base	NUMERIC(12,2)	—	No	Costo base	—	—	—
disponible	BOOLEAN	—	No	Disponible	—	—	—

espacio_hospitalario

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
espacio_hospitalario_id	BIGINT	PK	No	Identificador único del registro	—	—	—
hospital_id	BIGINT	FK	No	Referencia a hospital	RESTRICT	CASCADE	Conserva el registro relacionado de hospital y su historial.
unidad_medica_id	BIGINT	FK	Sí	Referencia a unidad_medica	SET NULL	CASCADE	Permite retirar la referencia opcional a unidad_medica.
codigo	VARCHAR(20)	—	No	Codigo	—	—	—
tipo	VARCHAR(15)	—	No	Tipo	—	—	—
nombre	VARCHAR(100)	—	Sí	Nombre	—	—	—
estado	VARCHAR(20)	—	No	Estado	—	—	—
costo_diario	NUMERIC(12,2)	—	Sí	Costo diario	—	—	—

institucion_externa

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
institucion_externa_id	BIGINT	PK	No	Identificador único del registro	—	—	—

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
direccion_id	BIGINT	FK	Sí	Referencia a direccion	SET NULL	CASCADE	Permite retirar la referencia opcional a direccion.
nombre	VARCHAR(150)	—	No	Nombre	—	—	—
tipo	VARCHAR(30)	—	No	Tipo	—	—	—
telefono	VARCHAR(20)	—	Sí	Telefono	—	—	—

Personas y personal

Persona

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
persona_id	BIGINT	PK	No	Identificador único del registro	—	—	—
direccion_id	BIGINT	FK	Sí	Referencia a direccion	SET NULL	CASCADE	Permite retirar la referencia opcional a direccion.
nombres	VARCHAR(100)	—	No	Nombres	—	—	—
apellidos	VARCHAR(100)	—	No	Apellidos	—	—	—
dpi	CHAR(13)	—	Sí	Dpi	—	—	—
fecha_nacimiento	DATE	—	No	Fecha nacimiento	—	—	—
sexo	VARCHAR(10)	—	No	Sexo	—	—	—
estado_civil	VARCHAR(15)	—	Sí	Estado civil	—	—	—
telefono	VARCHAR(20)	—	Sí	Telefono	—	—	—
correo	VARCHAR(120)	—	Sí	Correo	—	—	—

Paciente

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
paciente_id	BIGINT	PK/FK	No	Identificador único del registro	RESTRICT	CASCADE	Conserva el registro relacionado de persona y su historial.
numero_expediente	VARCHAR(25)	—	No	Numero expediente	—	—	—
numero_seguro_social	VARCHAR(30)	—	Sí	Numero seguro social	—	—	—

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
fecha_registro	DATE	—	No	Fecha registro	—	—	—
activo	BOOLEAN	—	No	Activo	—	—	—

encargado_paciente

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
encargado_paciente_id	BIGINT	PK	No	Identificador único del registro	—	—	—
paciente_id	BIGINT	FK	No	Referencia a paciente	RESTRICT	CASCADE	Conserva el registro relacionado de paciente y su historial.
encargado_persona_id	BIGINT	FK	No	Referencia a persona	RESTRICT	CASCADE	Conserva el registro relacionado de persona y su historial.
parentesco	VARCHAR(40)	—	No	Parentesco	—	—	—
es_principal	BOOLEAN	—	No	Es principal	—	—	—
activo	BOOLEAN	—	No	Activo	—	—	—

Empleado

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
empleado_id	BIGINT	PK	No	Identificador único del registro	—	—	—
persona_id	BIGINT	FK	No	Referencia a persona	RESTRICT	CASCADE	Conserva el registro relacionado de persona y su historial.
hospital_id	BIGINT	FK	No	Referencia a hospital	RESTRICT	CASCADE	Conserva el registro relacionado de hospital y su historial.
codigo_empleado	VARCHAR(20)	—	No	Codigo empleado	—	—	—
tipo_empleado	VARCHAR(25)	—	No	Tipo empleado	—	—	—
fecha_contratacion	DATE	—	No	Fecha contratacion	—	—	—
fecha_finalizacion	DATE	—	Sí	Fecha finalizacion	—	—	—
estado_laboral	VARCHAR(15)	—	No	Estado laboral	—	—	—

Especialidad

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
especialidad_id	SMALLINT	PK	No	Identificador único del registro	—	—	—
Nombre	VARCHAR(120)	—	No	Nombre	—	—	—
descripcion	VARCHAR(300)	—	Sí	Descripcion	—	—	—
activa	BOOLEAN	—	No	Activa	—	—	—

Medico

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
medico_id	BIGINT	PK/FK	No	Identificador único del registro	RESTRICT	CASCADE	Conserva el registro relacionado de persona y su historial.
numero_colegiado	VARCHAR(25)	—	No	Numero colegiado	—	—	—
clase_medico	VARCHAR(12)	—	No	Clase medico	—	—	—
institucion_externa_id	BIGINT	FK	Sí	Referencia a institucion_externa	RESTRICT	CASCADE	Conserva el registro relacionado de institucion_externa y su historial.
activo	BOOLEAN	—	No	Activo	—	—	—

medico_especialidad

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
medico_id	BIGINT	PK/FK	No	Identificador único del registro	CASCADE	CASCADE	El detalle depende del registro de medico.
especialidad_id	SMALLINT	PK/FK	No	Identificador único del registro	RESTRICT	CASCADE	Conserva el registro relacionado de especialidad y su historial.
es_principal	BOOLEAN	—	No	Es principal	—	—	—

medico_hospital

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
medico_hospital_id	BIGINT	PK	No	Identificador único del registro	—	—	—
medico_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
hospital_id	BIGINT	FK	No	Referencia a hospital	RESTRICT	CASCADE	Conserva el registro relacionado de hospital y su historial.
condicion	VARCHAR(12)	—	No	Condicion	—	—	—
fecha_inicio	DATE	—	No	Fecha inicio	—	—	—
fecha_fin	DATE	—	Sí	Fecha fin	—	—	—
activo	BOOLEAN	—	No	Activo	—	—	—

Enfermero

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
enfermero_id	BIGINT	PK/FK	No	Identificador único del registro	RESTRICT	CASCADE	Conserva el registro relacionado de empleado y su historial.
numero_registro	VARCHAR(25)	—	Sí	Numero registro	—	—	—
nivel	VARCHAR(12)	—	No	Nivel	—	—	—

practicante_medicina

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
practicante_id	BIGINT	PK/FK	No	Identificador único del registro	RESTRICT	CASCADE	Conserva el registro relacionado de empleado y su historial.
institucion_educativa	VARCHAR(150)	—	No	Institucion educativa	—	—	—
numero_carnet	VARCHAR(30)	—	No	Numero carnet	—	—	—
fecha_inicio	DATE	—	No	Fecha inicio	—	—	—
fecha_fin	DATE	—	No	Fecha fin	—	—	—

turno_personal

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
turno_personal_id	BIGINT	PK	No	Identificador único del registro	—	—	—

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
empleado_id	BIGINT	FK	No	Referencia a empleado	RESTRICT	CASCADE	Conserva el registro relacionado de empleado y su historial.
unidad_medica_id	BIGINT	FK	No	Referencia a unidad_medica	RESTRICT	CASCADE	Conserva el registro relacionado de unidad_medica y su historial.
fecha	DATE	—	No	Fecha	—	—	—
hora_inicio	TIME	—	No	Hora inicio	—	—	—
hora_fin	TIME	—	No	Hora fin	—	—	—
estado	VARCHAR(12)	—	No	Estado	—	—	—

Expediente clínico

episodio_atencion

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
episodio_atencion_id	BIGINT	PK	No	Identificador único del registro	—	—	—
paciente_id	BIGINT	FK	No	Referencia a paciente	RESTRICT	CASCADE	Conserva el registro relacionado de paciente y su historial.
hospital_id	BIGINT	FK	No	Referencia a hospital	RESTRICT	CASCADE	Conserva el registro relacionado de hospital y su historial.
fecha_apertura	TIMESTAMPTZ	—	No	Fecha apertura	—	—	—
fecha_cierre	TIMESTAMPTZ	—	Sí	Fecha cierre	—	—	—
estado	VARCHAR(12)	—	No	Estado	—	—	—
motivo_apertura	TEXT	—	No	Motivo apertura	—	—	—

Ingreso

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
ingreso_id	BIGINT	PK	No	Identificador único del registro	—	—	—
episodio_atencion_id	BIGINT	FK	No	Referencia a episodio_atencion	RESTRICT	CASCADE	Conserva el registro relacionado de episodio_atencion y su historial.

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
hospital_id	BIGINT	FK	No	Referencia a episodio_atencion, espacio_hospitalario, unidad_medica	RESTRICT	CASCADE	Conserva el registro relacionado de episodio_atencion y su historial.; Conserva el registro relacionado de unidad_medica y su historial.; Conserva el registro relacionado de espacio_hospitalario y su historial.
unidad_medica_id	BIGINT	FK	No	Referencia a unidad_medica, unidad_servicio	RESTRICT	CASCADE	Conserva el registro relacionado de unidad_medica y su historial.; Conserva el registro relacionado de unidad_servicio y su historial.
servicio_medico_id	SMALLINT	FK	No	Referencia a unidad_servicio	RESTRICT	CASCADE	Conserva el registro relacionado de unidad_servicio y su historial.
medico_encargado_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.
encargado_paciente_id	BIGINT	FK	Sí	Referencia a encargado_paciente	RESTRICT	CASCADE	Conserva el registro relacionado de encargado_paciente y su historial.
espacio_hospitalario_id	BIGINT	FK	Sí	Referencia a espacio_hospitalario	RESTRICT	CASCADE	Conserva el registro relacionado de espacio_hospitalario y su historial.
fecha_ingreso	TIMESTAMPZ	—	No	Fecha ingreso	—	—	—
motivo_ingreso	TEXT	—	No	Motivo ingreso	—	—	—
diagnostico_presuntivo	TEXT	—	No	Diagnostico presuntivo	—	—	—
estado	VARCHAR(10)	—	No	Estado	—	—	—

Diagnostico

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
diagnostico_id	BIGINT	PK	No	Identificador único del registro	—	—	—
codigo	VARCHAR(25)	—	Sí	Codigo	—	—	—
nombre	VARCHAR(200)	—	No	Nombre	—	—	—
descripcion	TEXT	—	Sí	Descripcion	—	—	—

Egreso

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
egreso_id	BIGINT	PK	No	Identificador único del registro	—	—	—

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
ingreso_id	BIGINT	FK	No	Referencia a ingreso	RESTRICT	CASCADE	Conserva el registro relacionado de ingreso y su historial.
medico_asignado_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.
institucion_referida_id	BIGINT	FK	Sí	Referencia a institucion_externa	RESTRICT	CASCADE	Conserva el registro relacionado de institucion_externa y su historial.
fecha_egreso	TIMESTAMPZ	—	No	Fecha egreso	—	—	—
motivo_egreso	TEXT	—	No	Motivo egreso	—	—	—
codigo_egreso	VARCHAR(10)	—	No	Codigo egreso	—	—	—
sin_consentimiento_medico	BOOLEAN	—	No	sin_consentimiento_medico	—	—	—
motivo_sin_consentimiento	TEXT	—	Sí	motivo_sin_consentimiento	—	—	—
operaciones_intervenciones	TEXT	—	Sí	operaciones_intervenciones	—	—	—
codigo_traslado	VARCHAR(40)	—	Sí	Codigo traslado	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—

egreso_diagnostico

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
egreso_id	BIGINT	PK/FK	No	Identificador único del registro	CASCADE	CASCADE	El detalle depende del registro de egreso.
diagnostico_id	BIGINT	PK/FK	No	Identificador único del registro	RESTRICT	CASCADE	Conserva el registro relacionado de diagnostico y su historial.
tipo	VARCHAR(10)	—	No	Tipo	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—

Traslado

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
traslado_id	BIGINT	PK	No	Identificador único del registro	—	—	—

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
episodio_atencion_id	BIGINT	FK	No	Referencia a episodio_atencion, ingreso	RESTRICT	CASCADE	Conserva el registro relacionado de episodio_atencion y su historial.; Conserva el registro relacionado de ingreso y su historial.
ingreso_origen_id	BIGINT	FK	Sí	Referencia a ingreso	RESTRICT	CASCADE	Conserva el registro relacionado de ingreso y su historial.
medico_indicador_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.
unidad_origen_id	BIGINT	FK	No	Referencia a unidad_medica	RESTRICT	CASCADE	Conserva el registro relacionado de unidad_medica y su historial.
unidad_destino_id	BIGINT	FK	Sí	Referencia a unidad_medica	RESTRICT	CASCADE	Conserva el registro relacionado de unidad_medica y su historial.
institucion_destino_id	BIGINT	FK	Sí	Referencia a institucion_externa	RESTRICT	CASCADE	Conserva el registro relacionado de institucion_externa y su historial.
servicio_destino_id	SMALLINT	FK	Sí	Referencia a servicio_medico	RESTRICT	CASCADE	Conserva el registro relacionado de servicio_medico y su historial.
fecha_traslado	TIMESTAMPZ	—	No	Fecha traslado	—	—	—
tipo	VARCHAR(10)	—	No	Tipo	—	—	—
motivo	TEXT	—	No	Motivo	—	—	—
codigo_traslado	VARCHAR(40)	—	No	Codigo traslado	—	—	—

consentimiento_general

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
consentimiento_general_ id	BIGINT	PK	No	Identificador único del registro	—	—	—
episodio_atencion_id	BIGINT	FK	No	Referencia a episodio_atencion	RESTRICT	CASCADE	Conserva el registro relacionado de episodio_atencion y su historial.
firmante_persona_id	BIGINT	FK	No	Referencia a persona	RESTRICT	CASCADE	Conserva el registro relacionado de persona y su historial.
medico_solicitante_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.
tipo	VARCHAR(17)	—	No	Tipo	—	—	—
fecha	TIMESTAMP TZ	—	No	Fecha	—	—	—

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
aceptado	BOOLEAN	—	No	Aceptado	—	—	—
descripcion_procedimiento	TEXT	—	No	Descripcion procedimiento	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—

Consulta y emergencia

horario_consulta

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
horario_consulta_id	BIGINT	PK	No	Identificador único del registro	—	—	—
medico_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.
unidad_medica_id	BIGINT	FK	No	Referencia a unidad_medica	RESTRICT	CASCADE	Conserva el registro relacionado de unidad_medica y su historial.
clinica_id	BIGINT	FK	No	Referencia a espacio_hospitalario	RESTRICT	CASCADE	Conserva el registro relacionado de espacio_hospitalario y su historial.
dia_semana	SMALLINT	—	No	Dia semana	—	—	—
hora_inicio	TIME	—	No	Hora inicio	—	—	—
hora_fin	TIME	—	No	Hora fin	—	—	—
activo	BOOLEAN	—	No	Activo	—	—	—

Cita

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
cita_id	BIGINT	PK	No	Identificador único del registro	—	—	—
paciente_id	BIGINT	FK	No	Referencia a paciente	RESTRICT	CASCADE	Conserva el registro relacionado de paciente y su historial.
medico_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.
unidad_medica_id	BIGINT	FK	No	Referencia a unidad_medica	RESTRICT	CASCADE	Conserva el registro relacionado de unidad_medica y su historial.

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
clinica_id	BIGINT	FK	No	Referencia a espacio_hospitalario	RESTRICT	CASCADE	Conserva el registro relacionado de espacio_hospitalario y su historial.
cita_anterior_id	BIGINT	FK	Sí	Referencia a cita	RESTRICT	CASCADE	Conserva el registro relacionado de cita y su historial.
fecha_hora	TIMESTAMP TZ	—	No	Fecha hora	—	—	—
canal	VARCHAR(10)	—	No	Canal	—	—	—
tipo	VARCHAR(12)	—	No	Tipo	—	—	—
estado	VARCHAR(15)	—	No	Estado	—	—	—
precio_base	NUMERIC(12,2)	—	No	Precio base	—	—	—
aplica_recargo_cancelacion	BOOLEAN	—	No	Aplica recargo cancelacion	—	—	—
motivo_cancelacion	TEXT	—	Sí	Motivo cancelacion	—	—	—
fecha_registro	TIMESTAMP TZ	—	No	Fecha registro	—	—	—

historial_estado_cita

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
historial_estado_cita_id	BIGINT	PK	No	Identificador único del registro	—	—	—
cita_id	BIGINT	FK	No	Referencia a cita	CASCADE	CASCADE	El detalle depende del registro de cita.
estado_anterior	VARCHAR(15)	—	Sí	Estado anterior	—	—	—
estado_nuevo	VARCHAR(15)	—	No	Estado nuevo	—	—	—
fecha_cambio	TIMESTAMPTZ	—	No	Fecha cambio	—	—	—
motivo	TEXT	—	Sí	Motivo	—	—	—

consulta_externa

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
consulta_externa_id	BIGINT	PK	No	Identificador único del registro	—	—	—
cita_id	BIGINT	FK	No	Referencia a cita	RESTRICT	CASCADE	Conserva el registro relacionado de cita y su historial.

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
fecha_atencion	TIMESTAMPZ	—	No	Fecha atencion	—	—	—
diagnostico	TEXT	—	No	Diagnostico	—	—	—
datos_interes	TEXT	—	Sí	Datos interes	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—
orientacion_paciente	TEXT	—	Sí	Orientacion paciente	—	—	—

Receta

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
receta_id	BIGINT	PK	No	Identificador único del registro	—	—	—
consulta_externa_id	BIGINT	FK	No	Referencia a consulta_externa	RESTRICT	CASCADE	Conserva el registro relacionado de consulta_externa y su historial.
fecha_emision	DATE	—	No	Fecha emision	—	—	—
fecha_proxima_cita	DATE	—	Sí	Fecha proxima cita	—	—	—
indicaciones	TEXT	—	Sí	Indicaciones	—	—	—

Medicamento

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
medicamento_id	BIGINT	PK	No	Identificador único del registro	—	—	—
nombre	VARCHAR(120)	—	No	Nombre	—	—	—
presentacion	VARCHAR(100)	—	No	Presentacion	—	—	—
activo	BOOLEAN	—	No	Activo	—	—	—

receta_detalle

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
receta_detalle_id	BIGINT	PK	No	Identificador único del registro	—	—	—
receta_id	BIGINT	FK	No	Referencia a receta	CASCADE	CASCADE	El detalle depende del registro de receta.

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
medicamento_id	BIGINT	FK	No	Referencia a medicamento	RESTRICT	CASCADE	Conserva el registro relacionado de medicamento y su historial.
dosis	VARCHAR(100)	—	No	Dosis	—	—	—
frecuencia	VARCHAR(100)	—	No	Frecuencia	—	—	—
duracion_dias	SMALLINT	—	No	Duracion días	—	—	—
instrucciones	TEXT	—	Sí	Instrucciones	—	—	—

orden_laboratorio

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
orden_laboratorio_id	BIGINT	PK	No	Identificador único del registro	—	—	—
consulta_externa_id	BIGINT	FK	No	Referencia a consulta_externa	RESTRICT	CASCADE	Conserva el registro relacionado de consulta_externa y su historial.
fecha_emision	TIMESTAMPZ	—	No	Fecha emision	—	—	—
indicaciones	TEXT	—	Sí	Indicaciones	—	—	—
estado	VARCHAR(12)	—	No	Estado	—	—	—

orden_laboratorio_detalle

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
orden_laboratorio_detalle_id	BIGINT	PK	No	Identificador único del registro	—	—	—
orden_laboratorio_id	BIGINT	FK	No	Referencia a orden_laboratorio	CASCADE	CASCADE	El detalle depende del registro de orden_laboratorio.
examen	VARCHAR(150)	—	No	Examen	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—

atencion_emergencia

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
atencion_emergencia_id	BIGINT	PK	No	Identificador único del registro	—	—	—
ingreso_id	BIGINT	FK	No	Referencia a ingreso	RESTRICT	CASCADE	Conserva el registro relacionado de ingreso y su historial.
fecha_evaluacion	TIMESTAMPZ	—	No	Fecha evaluacion	—	—	—
prioridad	SMALLINT	—	No	Prioridad	—	—	—
estado_clinico	TEXT	—	No	Estado clinico	—	—	—
resultado	TEXT	—	Sí	Resultado	—	—	—

procedimiento_emergencia

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
procedimiento_emergencia_id	BIGINT	PK	No	Identificador único del registro	—	—	—
nombre	VARCHAR(180)	—	No	Nombre	—	—	—
tipo	VARCHAR(15)	—	No	Tipo	—	—	—
descripcion	TEXT	—	Sí	Descripcion	—	—	—
costo_base	NUMERIC(12,2)	—	No	Costo base	—	—	—

emergencia_procedimiento

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
emergencia_procedimiento_id	BIGINT	PK	No	Identificador único del registro	—	—	—
atencion_emergencia_id	BIGINT	FK	No	Referencia a atencion_emergencia	RESTRICT	CASCADE	Conserva el registro relacionado de atencion_emergencia y su historial.
procedimiento_emergencia_id	BIGINT	FK	No	Referencia a procedimiento_emergencia	RESTRICT	CASCADE	Conserva el registro relacionado de procedimiento_emergencia y su historial.
medico_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.
fecha_realizacion	TIMESTAMPTZ	—	No	Fecha realizacion	—	—	—
resultado	TEXT	—	Sí	Resultado	—	—	—
costo_aplicado	NUMERIC(12,2)	—	No	Costo aplicado	—	—	—

cola_emergencia

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
cola_emergencia_id	BIGINT	PK	No	Identificador único del registro	—	—	—
atencion_emergencia_id	BIGINT	FK	No	Referencia a atencion_emergencia	RESTRICT	CASCADE	Conserva el registro relacionado de atencion_emergencia y su historial.
fecha_inicio	TIMESTAMPTZ	—	No	Fecha inicio	—	—	—
fecha_fin	TIMESTAMPTZ	—	Sí	Fecha fin	—	—	—
prioridad	SMALLINT	—	No	Prioridad	—	—	—
estado	VARCHAR(12)	—	No	Estado	—	—	—

Cirugía

solicitud_cirugia

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
solicitud_cirugia_id	BIGINT	PK	No	Identificador único del registro	—	—	—
paciente_id	BIGINT	FK	No	Referencia a paciente	RESTRICT	CASCADE	Conserva el registro relacionado de paciente y su historial.
hospital_id	BIGINT	FK	No	Referencia a episodio_atencion, hospital	RESTRICT	CASCADE	Conserva el registro relacionado de hospital y su historial.; Conserva el registro relacionado de episodio_atencion y su historial.
cirujano_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.
episodio_atencion_id	BIGINT	FK	Sí	Referencia a episodio_atencion	RESTRICT	CASCADE	Conserva el registro relacionado de episodio_atencion y su historial.
fecha_solicitud	TIMESTAMPZ	—	No	Fecha solicitud	—	—	—
caracter	VARCHAR(10)	—	No	Caracter	—	—	—
historia_clinica	TEXT	—	No	Historia clinica	—	—	—
procedimiento_propuesto	TEXT	—	No	Procedimiento propuesto	—	—	—

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
duracion_estimada_minutos	INTEGER	—	No	Duracion estimada minutos	—	—	—
tipo_anestesia	VARCHAR(80)	—	No	Tipo anestesia	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—

evaluacion_solicitud_cirugia

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
evaluacion_solicitud_cirugia_id	BIGINT	PK	No	Identificador único del registro	—	—	—
solicitud_cirugia_id	BIGINT	FK	No	Referencia a solicitud_cirugia	RESTRICT	CASCADE	Conserva el registro relacionado de solicitud_cirugia y su historial.
medico_evaluador_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.
fecha_evaluacion	TIMESTAMP TZ	—	No	Fecha evaluacion	—	—	—
decision	VARCHAR(10)	—	No	Decision	—	—	—
razones	TEXT	—	Sí	Razones	—	—	—

Cirugia

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
cirugia_id	BIGINT	PK	No	Identificador único del registro	—	—	—
solicitud_cirugia_id	BIGINT	FK	No	Referencia a evaluacion_solicitud_cirugia, solicitud_cirugia	RESTRICT	CASCADE	Conserva el registro relacionado de solicitud_cirugia y su historial.; Conserva el registro relacionado de evaluacion_solicitud_cirugia y su historial.
evaluacion_solicitud_cirugia_id	BIGINT	FK	No	Referencia a evaluacion_solicitud_cirugia	RESTRICT	CASCADE	Conserva el registro relacionado de evaluacion_solicitud_cirugia y su historial.
decision_requerida	VARCHAR(10)	FK	No	Referencia a evaluacion_solicitud_cirugia	RESTRICT	CASCADE	Conserva el registro relacionado de evaluacion_solicitud_cirugia y su historial.
quirofano_id	BIGINT	FK	No	Referencia a espacio_hospitalario	RESTRICT	CASCADE	Conserva el registro relacionado de espacio_hospitalario y su historial.
fecha_programada	TIMESTAMP TZ	—	No	Fecha programada	—	—	—

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
fecha_inicio	TIMESTAMPZ	—	Sí	Fecha inicio	—	—	—
fecha_fin	TIMESTAMPZ	—	Sí	Fecha fin	—	—	—
estado	VARCHAR(12)	—	No	Estado	—	—	—
procedimiento_realizado	TEXT	—	Sí	Procedimiento realizado	—	—	—

participante_cirugia

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
participante_cirugia_id	BIGINT	PK	No	Identificador único del registro	—	—	—
cirugia_id	BIGINT	FK	No	Referencia a cirugia	RESTRICT	CASCADE	Conserva el registro relacionado de cirugia y su historial.
persona_id	BIGINT	FK	No	Referencia a persona	RESTRICT	CASCADE	Conserva el registro relacionado de persona y su historial.
rol	VARCHAR(25)	—	No	Rol	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—

Recurso

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
recurso_id	BIGINT	PK	No	Identificador único del registro	—	—	—
nombre	VARCHAR(150)	—	No	Nombre	—	—	—
descripcion	TEXT	—	Sí	Descripcion	—	—	—
categoria	VARCHAR(11)	—	No	Categoria	—	—	—
tipo	VARCHAR(12)	—	No	Tipo	—	—	—
material	VARCHAR(100)	—	Sí	Material	—	—	—
funcion	VARCHAR(20)	—	Sí	Funcion	—	—	—
costo_unitario	NUMERIC(12,2)	—	No	Costo unitario	—	—	—
activo	BOOLEAN	—	No	Activo	—	—	—

solicitud_recurso

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
solicitud_cirugia_id	BIGINT	PK/FK	No	Identificador único del registro	RESTRICT	CASCADE	Conserva el registro relacionado de solicitud_cirugia y su historial.
recurso_id	BIGINT	PK/FK	No	Identificador único del registro	RESTRICT	CASCADE	Conserva el registro relacionado de recurso y su historial.
cantidad	NUMERIC(10,2)	—	No	Cantidad	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—

uso_recurso_cirugia

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
uso_recurso_cirugia_id	BIGINT	PK	No	Identificador único del registro	—	—	—
cirugia_id	BIGINT	FK	No	Referencia a cirugia	RESTRICT	CASCADE	Conserva el registro relacionado de cirugia y su historial.
recurso_id	BIGINT	FK	No	Referencia a recurso	RESTRICT	CASCADE	Conserva el registro relacionado de recurso y su historial.
cantidad	NUMERIC(10,2)	—	No	Cantidad	—	—	—
costo_unitario_aplicado	NUMERIC(12,2)	—	No	Costo unitario aplicado	—	—	—
fecha_uso	TIMESTAMPTZ	—	No	Fecha uso	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—

consentimiento_quirurgico

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
consentimiento_quirurgico_id	BIGINT	PK	No	Identificador único del registro	—	—	—
cirugia_id	BIGINT	FK	No	Referencia a cirugia	RESTRICT	CASCADE	Conserva el registro relacionado de cirugia y su historial.
medico_firmante_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.
firmante_persona_id	BIGINT	FK	No	Referencia a persona	RESTRICT	CASCADE	Conserva el registro relacionado de persona y su historial.

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
calidad_firmante	VARCHAR(20)	—	No	Calidad firmante	—	—	—
procedimiento	TEXT	—	No	Procedimiento	—	—	—
objetivo	TEXT	—	No	Objetivo	—	—	—
caracteristicas	TEXT	—	No	Caracteristicas	—	—	—
riesgos	TEXT	—	No	Riesgos	—	—	—
fecha_obtencion	TIMESTAMPTZ	—	No	Fecha obtencion	—	—	—
firma_medico_confirmada	BOOLEAN	—	No	Firma medico confirmada	—	—	—
firma_paciente_o_representante_confirmada	BOOLEAN	—	No	Firma paciente o representante confirmada	—	—	—

chequeo_preadnestesico

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
chequeo_preadnestesico_id	BIGINT	PK	No	Identificador único del registro	—	—	—
cirugia_id	BIGINT	FK	No	Referencia a cirugia	RESTRICT	CASCADE	Conserva el registro relacionado de cirugia y su historial.
medico_evaluador_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.
anestesista_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.
clasificacion_asa	VARCHAR(3)	—	No	Clasificacion asa	—	—	—
plan_anestesia	TEXT	—	No	Plan anestesia	—	—	—
fecha_evaluacion	TIMESTAMPZ	—	No	Fecha evaluacion	—	—	—
firma_evaluador_confirmada	BOOLEAN	—	No	Firma evaluador confirmada	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—

etapa_cirugia

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
etapa_cirugia_id	BIGINT	PK	No	Identificador único del registro	—	—	—
cirugia_id	BIGINT	FK	No	Referencia a cirugia	RESTRICT	CASCADE	Conserva el registro relacionado de cirugia y su historial.
enfermero_responsable_id	BIGINT	FK	No	Referencia a enfermero	RESTRICT	CASCADE	Conserva el registro relacionado de enfermero y su historial.
etapa	VARCHAR(16)	—	No	Etapas	—	—	—
fecha_inicio	TIMESTAMPZ	—	No	Fecha inicio	—	—	—
fecha_fin	TIMESTAMPZ	—	Sí	Fecha fin	—	—	—
fecha_envio_secretaria	TIMESTAMPZ	—	Sí	Fecha envío secretaria	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—

item_control_quirurgico

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
item_control_quirurgico_id	BIGINT	PK	No	Identificador único del registro	—	—	—
etapa	VARCHAR(16)	—	No	Etapas	—	—	—
grupo	VARCHAR(40)	—	No	Grupo	—	—	—
descripcion	VARCHAR(250)	—	No	Descripcion	—	—	—
tipo_resultado	VARCHAR(12)	—	No	Tipo resultado	—	—	—
activo	BOOLEAN	—	No	Activo	—	—	—

resultado_control_quirurgico

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
resultado_control_quirurgico_id	BIGINT	PK	No	Identificador único del registro	—	—	—
etapa_cirugia_id	BIGINT	FK	No	Referencia a etapa_cirugia	RESTRICT	CASCADE	Conserva el registro relacionado de etapa_cirugia y su historial.
item_control_quirurgico_id	BIGINT	FK	No	Referencia a item_control_quirurgico	RESTRICT	CASCADE	Conserva el registro relacionado de item_control_quirurgico y su historial.

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
etapa	VARCHAR(16)	FK	No	Referencia a etapa_cirugia, item_control_quirurgico	RESTRICT	CASCADE	Conserva el registro relacionado de etapa_cirugia y su historial.; Conserva el registro relacionado de item_control_quirurgico y su historial.
resultado	VARCHAR(25)	—	No	Resultado	—	—	—
fecha_registro	TIMESTAMPZ	—	No	Fecha registro	—	—	—
registrado_por_enfermero_id	BIGINT	FK	No	Referencia a enfermero	RESTRICT	CASCADE	Conserva el registro relacionado de enfermero y su historial.
observaciones	TEXT	—	Sí	Observaciones	—	—	—

registro_clinico_cirugia

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
registro_clinico_cirugia_id	BIGINT	PK	No	Identificador único del registro	—	—	—
cirugia_id	BIGINT	FK	No	Referencia a cirugia	RESTRICT	CASCADE	Conserva el registro relacionado de cirugia y su historial.
etapa_cirugia_id	BIGINT	FK	Sí	Referencia a etapa_cirugia	RESTRICT	CASCADE	Conserva el registro relacionado de etapa_cirugia y su historial.
registrador_persona_id	BIGINT	FK	No	Referencia a persona	RESTRICT	CASCADE	Conserva el registro relacionado de persona y su historial.
categoria	VARCHAR(32)	—	No	Categoria	—	—	—
descripcion	TEXT	—	No	Descripcion	—	—	—
valor	TEXT	—	Sí	Valor	—	—	—
fecha_registro	TIMESTAMPZ	—	No	Fecha registro	—	—	—

Hospitalización

Hospitalizacion

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
hospitalizacion_id	BIGINT	PK	No	Identificador único del registro	—	—	—
ingreso_id	BIGINT	FK	No	Referencia a ingreso	RESTRICT	CASCADE	Conserva el registro relacionado de ingreso y su historial.
medico_encargado_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.
fecha_alta_prevista	TIMESTAMPTZ	—	Sí	Fecha alta prevista	—	—	—
fecha_alta_real	TIMESTAMPTZ	—	Sí	Fecha alta real	—	—	—
costo_diario_aplicado	NUMERIC(12,2)	—	No	Costo diario aplicado	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—

asignacion_cama

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
asignacion_cama_id	BIGINT	PK	No	Identificador único del registro	—	—	—
hospitalizacion_id	BIGINT	FK	No	Referencia a hospitalizacion	RESTRICT	CASCADE	Conserva el registro relacionado de hospitalizacion y su historial.
cama_id	BIGINT	FK	No	Referencia a espacio_hospitalario	RESTRICT	CASCADE	Conserva el registro relacionado de espacio_hospitalario y su historial.
fecha_inicio	TIMESTAMPTZ	—	No	Fecha inicio	—	—	—
fecha_fin	TIMESTAMPTZ	—	Sí	Fecha fin	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—

hospitalizacion_servicio

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
hospitalizacion_servicio_id	BIGINT	PK	No	Identificador único del registro	—	—	—
hospitalizacion_id	BIGINT	FK	No	Referencia a hospitalizacion	RESTRICT	CASCADE	Conserva el registro relacionado de hospitalizacion y su historial.
servicio_medico_id	SMALLINT	FK	No	Referencia a servicio_medico	RESTRICT	CASCADE	Conserva el registro relacionado de servicio_medico y su historial.
medico_id	BIGINT	FK	No	Referencia a medico	RESTRICT	CASCADE	Conserva el registro relacionado de medico y su historial.

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
fecha_servicio	TIMESTAMPZ	—	No	Fecha servicio	—	—	—
descripcion	TEXT	—	No	Descripcion	—	—	—
costo_aplicado	NUMERIC(12,2)	—	No	Costo aplicado	—	—	—

hospitalizacion_insumo

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
hospitalizacion_insumo_id	BIGINT	PK	No	Identificador único del registro	—	—	—
hospitalizacion_id	BIGINT	FK	No	Referencia a hospitalizacion	RESTRICT	CASCADE	Conserva el registro relacionado de hospitalizacion y su historial.
recurso_id	BIGINT	FK	No	Referencia a recurso	RESTRICT	CASCADE	Conserva el registro relacionado de recurso y su historial.
fecha_uso	TIMESTAMPZ	—	No	Fecha uso	—	—	—
cantidad	NUMERIC(10,2)	—	No	Cantidad	—	—	—
costo_unitario_aplicado	NUMERIC(12,2)	—	No	Costo unitario aplicado	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—

Facturación y calidad

Factura

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
factura_id	BIGINT	PK	No	Identificador único del registro	—	—	—
numero_factura	VARCHAR(30)	—	No	Numero factura	—	—	—
paciente_id	BIGINT	FK	No	Referencia a paciente	RESTRICT	CASCADE	Conserva el registro relacionado de paciente y su historial.
hospital_id	BIGINT	FK	No	Referencia a hospital	RESTRICT	CASCADE	Conserva el registro relacionado de hospital y su historial.
ingreso_id	BIGINT	FK	Sí	Referencia a ingreso	RESTRICT	CASCADE	Conserva el registro relacionado de ingreso y su historial.

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
consulta_externa_id	BIGINT	FK	Sí	Referencia a consulta_externa	RESTRICT	CASCADE	Conserva el registro relacionado de consulta_externa y su historial.
fecha_emision	TIMESTAMPZ	—	No	Fecha emision	—	—	—
descripcion	VARCHAR(300)	—	No	Descripcion	—	—	—
estado	VARCHAR(12)	—	No	Estado	—	—	—

factura_detalle

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
factura_detalle_id	BIGINT	PK	No	Identificador único del registro	—	—	—
factura_id	BIGINT	FK	No	Referencia a factura	RESTRICT	CASCADE	Conserva el registro relacionado de factura y su historial.
concepto	VARCHAR(250)	—	No	Concepto	—	—	—
cantidad	NUMERIC(10,2)	—	No	Cantidad	—	—	—
precio_unitario	NUMERIC(12,2)	—	No	Precio unitario	—	—	—
porcentaje_descuento	NUMERIC(5,2)	—	No	Porcentaje descuento	—	—	—
porcentaje_recargo	NUMERIC(5,2)	—	No	Porcentaje recargo	—	—	—
subtotal	NUMERIC(14,2)	—	Sí	Subtotal	—	—	—

plan_pago

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
plan_pago_id	BIGINT	PK	No	Identificador único del registro	—	—	—
factura_id	BIGINT	FK	No	Referencia a factura	RESTRICT	CASCADE	Conserva el registro relacionado de factura y su historial.
numero_cuotas	SMALLINT	—	No	Numero cuotas	—	—	—
fecha_inicio	DATE	—	No	Fecha inicio	—	—	—
estado	VARCHAR(12)	—	No	Estado	—	—	—

Cuota

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
cuota_id	BIGINT	PK	No	Identificador único del registro	—	—	—
plan_pago_id	BIGINT	FK	No	Referencia a plan_pago	RESTRICT	CASCADE	Conserva el registro relacionado de plan_pago y su historial.
numero	SMALLINT	—	No	Numero	—	—	—
fecha_vencimiento	DATE	—	No	Fecha vencimiento	—	—	—
monto	NUMERIC(12,2)	—	No	Monto	—	—	—
estado	VARCHAR(12)	—	No	Estado	—	—	—

Pago

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
pago_id	BIGINT	PK	No	Identificador único del registro	—	—	—
factura_id	BIGINT	FK	No	Referencia a factura, plan_pago	RESTRICT	CASCADE	Conserva el registro relacionado de factura y su historial.; Conserva el registro relacionado de plan_pago y su historial.
plan_pago_id	BIGINT	FK	Sí	Referencia a cuota, plan_pago	RESTRICT	CASCADE	Conserva el registro relacionado de plan_pago y su historial.; Conserva el registro relacionado de cuota y su historial.
cuota_id	BIGINT	FK	Sí	Referencia a cuota	RESTRICT	CASCADE	Conserva el registro relacionado de cuota y su historial.
unidad_recepcion_id	BIGINT	FK	No	Referencia a unidad_medica	RESTRICT	CASCADE	Conserva el registro relacionado de unidad_medica y su historial.
fecha_pago	TIMESTAMPZ	—	No	Fecha pago	—	—	—
monto	NUMERIC(12,2)	—	No	Monto	—	—	—
metodo	VARCHAR(15)	—	No	Metodo	—	—	—
referencia	VARCHAR(80)	—	Sí	Referencia	—	—	—

evaluacion_calidad

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
evaluacion_calidad_id	BIGINT	PK	No	Identificador único del registro	—	—	—
paciente_id	BIGINT	FK	No	Referencia a paciente	RESTRICT	CASCADE	Conserva el registro relacionado de paciente y su historial.
hospital_id	BIGINT	FK	No	Referencia a hospital	RESTRICT	CASCADE	Conserva el registro relacionado de hospital y su historial.
ingreso_id	BIGINT	FK	Sí	Referencia a ingreso	RESTRICT	CASCADE	Conserva el registro relacionado de ingreso y su historial.
consulta_externa_id	BIGINT	FK	Sí	Referencia a consulta_externa	RESTRICT	CASCADE	Conserva el registro relacionado de consulta_externa y su historial.
fecha_evaluacion	TIMESTAMPZ	—	No	Fecha evaluacion	—	—	—
observaciones	TEXT	—	Sí	Observaciones	—	—	—

evaluacion_objetivo

Atributo	Tipo de dato	Llave	Nulos	Descripción	Acción DELETE	Acción UPDATE	Justificación de la FK
evaluacion_objetivo_id	BIGINT	PK	No	Identificador único del registro	—	—	—
evaluacion_calidad_id	BIGINT	FK	No	Referencia a evaluacion_calidad	RESTRICT	CASCADE	Conserva el registro relacionado de evaluacion_calidad y su historial.
hospital_objetivo_id	BIGINT	FK	Sí	Referencia a hospital	RESTRICT	CASCADE	Conserva el registro relacionado de hospital y su historial.
persona_objetivo_id	BIGINT	FK	Sí	Referencia a persona	RESTRICT	CASCADE	Conserva el registro relacionado de persona y su historial.
nota	SMALLINT	—	No	Nota	—	—	—
comentario	TEXT	—	Sí	Comentario	—	—	—

4. Sentencias SQL

Se incluyen íntegramente los scripts ejecutados en PostgreSQL 16. Los archivos separados forman parte de la entrega y mantienen los comentarios de propósito y de validación.

estructura.sql

```
BEGIN;
```

```
CREATE SCHEMA IF NOT EXISTS hospital;
```

```
SET search_path TO hospital, public;
```

```
-- 1. ORGANIZACION Y UBICACION
```

```
-- Almacena el catalogo de departamentos de Guatemala.
```

```
CREATE TABLE IF NOT EXISTS departamento (  
    departamento_id SMALLINT GENERATED ALWAYS AS IDENTITY,  
    nombre VARCHAR(60) NOT NULL,  
    CONSTRAINT pk_departamento PRIMARY KEY (departamento_id),  
    CONSTRAINT uq_departamento_nombre UNIQUE (nombre),  
    -- Valida que el nombre no sea una cadena vacia.  
    CONSTRAINT ck_departamento_nombre_no_vacio  
        CHECK (BTRIM(nombre) <> '')  
);
```

```
-- Almacena los municipios y el departamento al que pertenecen.
```

```
CREATE TABLE IF NOT EXISTS municipio (  
    municipio_id INTEGER GENERATED ALWAYS AS IDENTITY,  
    departamento_id SMALLINT NOT NULL,  
    nombre VARCHAR(80) NOT NULL,  
    CONSTRAINT pk_municipio PRIMARY KEY (municipio_id),  
    CONSTRAINT uq_municipio_departamento_nombre  
        UNIQUE (departamento_id, nombre),  
    -- Valida que el nombre no sea una cadena vacia.  
    CONSTRAINT ck_municipio_nombre_no_vacio  
        CHECK (BTRIM(nombre) <> ''),  
    CONSTRAINT fk_municipio_departamento  
        FOREIGN KEY (departamento_id)  
        REFERENCES departamento (departamento_id)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT  
);
```

```

-- Centraliza las direcciones de hospitales, personas e instituciones.
CREATE TABLE IF NOT EXISTS direccion (
    direccion_id BIGINT GENERATED ALWAYS AS IDENTITY,
    municipio_id INTEGER NOT NULL,
    detalle VARCHAR(250) NOT NULL,
    zona VARCHAR(20),
    codigo_postal VARCHAR(10),
    area VARCHAR(10) NOT NULL,
    CONSTRAINT pk_direccion PRIMARY KEY (direccion_id),
    -- Valida que la direccion descriptiva no este vacia.
    CONSTRAINT ck_direccion_detalle_no_vacio
        CHECK (BTRIM(detalle) <> ''),
    -- Limita el area a los valores indicados en las fichas.
    CONSTRAINT ck_direccion_area
        CHECK (area IN ('Urbana', 'Rural')),
    CONSTRAINT fk_direccion_municipio
        FOREIGN KEY (municipio_id)
        REFERENCES municipio (municipio_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);

-- Registra los establecimientos de la cadena Hospitales de Occidente.
CREATE TABLE IF NOT EXISTS hospital (
    hospital_id BIGINT GENERATED ALWAYS AS IDENTITY,
    direccion_id BIGINT NOT NULL,
    codigo VARCHAR(15) NOT NULL,
    nombre VARCHAR(120) NOT NULL,
    telefono VARCHAR(20) NOT NULL,
    correo VARCHAR(120),
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_hospital PRIMARY KEY (hospital_id),
    CONSTRAINT uq_hospital_codigo UNIQUE (codigo),
    CONSTRAINT uq_hospital_nombre UNIQUE (nombre),
    -- Valida que el codigo institucional no este vacio.
    CONSTRAINT ck_hospital_codigo_no_vacio
        CHECK (BTRIM(codigo) <> ''),
    -- Valida que el nombre institucional no este vacio.
    CONSTRAINT ck_hospital_nombre_no_vacio
        CHECK (BTRIM(nombre) <> ''),
    -- Permite digitos, espacios, parentesis, signo mas y guiones en telefonos.

```

```

CONSTRAINT ck_hospital_telefono_formato

    CHECK (telefono ~ '^[+() 0-9]{8,20}$'),

-- Realiza una validacion basica del correo cuando se proporciona.

CONSTRAINT ck_hospital_correo_formato

    CHECK (correo IS NULL OR correo ~* '^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$'),

CONSTRAINT fk_hospital_direccion

    FOREIGN KEY (direccion_id)

    REFERENCES direccion (direccion_id)

    ON UPDATE CASCADE

    ON DELETE RESTRICT

);

-- Registra las cuatro unidades medicas que funcionan en cada hospital.

CREATE TABLE IF NOT EXISTS unidad_medica (

    unidad_medica_id BIGINT GENERATED ALWAYS AS IDENTITY,

    hospital_id BIGINT NOT NULL,

    nombre VARCHAR(100) NOT NULL,

    tipo VARCHAR(20) NOT NULL,

    ubicacion VARCHAR(150),

    telefono_extension VARCHAR(10),

    activa BOOLEAN NOT NULL DEFAULT TRUE,

    CONSTRAINT pk_unidad_medica PRIMARY KEY (unidad_medica_id),

    CONSTRAINT uq_unidad_medica_hospital_tipo UNIQUE (hospital_id, tipo),

    -- Limita las unidades a las cuatro definidas por el proyecto.

    CONSTRAINT ck_unidad_medica_tipo

        CHECK (tipo IN ('Consulta Externa', 'Emergencias', 'Cirugia', 'Hospitalizacion')),

    -- Valida que el nombre de la unidad no este vacio.

    CONSTRAINT ck_unidad_medica_nombre_no_vacio

        CHECK (BTRIM(nombre) <> ''),

    CONSTRAINT fk_unidad_medica_hospital

        FOREIGN KEY (hospital_id)

        REFERENCES hospital (hospital_id)

        ON UPDATE CASCADE

        ON DELETE RESTRICT

);

```

```

-- Cataloga los servicios y tipos de atencion medica ofrecidos.
CREATE TABLE IF NOT EXISTS servicio_medico (
    servicio_medico_id SMALLINT GENERATED ALWAYS AS IDENTITY,
    nombre VARCHAR(120) NOT NULL,
    categoria VARCHAR(25) NOT NULL,
    descripcion VARCHAR(300),
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_servicio_medico PRIMARY KEY (servicio_medico_id),
    CONSTRAINT uq_servicio_medico_nombre UNIQUE (nombre),
    -- Clasifica el servicio segun las unidades descritas en el enunciado.
    CONSTRAINT ck_servicio_medico_categoria
        CHECK (categoria IN ('Consulta Externa', 'Emergencias', 'Cirugia', 'Hospitalizacion', 'General')),
    -- Valida que el nombre del servicio no este vacio.
    CONSTRAINT ck_servicio_medico_nombre_no_vacio
        CHECK (BTRIM(nombre) <> '')
);

-- Resuelve la relacion muchos a muchos entre unidades y servicios.
CREATE TABLE IF NOT EXISTS unidad_servicio (
    unidad_medica_id BIGINT NOT NULL,
    servicio_medico_id SMALLINT NOT NULL,
    costo_base NUMERIC(12,2) NOT NULL DEFAULT 0,
    disponible BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_unidad_servicio
        PRIMARY KEY (unidad_medica_id, servicio_medico_id),
    -- Impide registrar costos negativos.
    CONSTRAINT ck_unidad_servicio_costo_base
        CHECK (costo_base >= 0),
    CONSTRAINT fk_unidad_servicio_unidad
        FOREIGN KEY (unidad_medica_id)
        REFERENCES unidad_medica (unidad_medica_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT fk_unidad_servicio_servicio
        FOREIGN KEY (servicio_medico_id)
        REFERENCES servicio_medico (servicio_medico_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);

```

```

-- Registra clinicas, camillas, quirofanos, salas y camas hospitalarias.
CREATE TABLE IF NOT EXISTS espacio_hospitalario (
    espacio_hospitalario_id BIGINT GENERATED ALWAYS AS IDENTITY,
    hospital_id BIGINT NOT NULL,
    unidad_medica_id BIGINT,
    codigo VARCHAR(20) NOT NULL,
    tipo VARCHAR(15) NOT NULL,
    nombre VARCHAR(100),
    estado VARCHAR(20) NOT NULL DEFAULT 'Disponible',
    costo_diario NUMERIC(12,2),
    CONSTRAINT pk_espacio_hospitalario PRIMARY KEY (espacio_hospitalario_id),
    CONSTRAINT uq_espacio_hospitalario_codigo UNIQUE (hospital_id, codigo),
    -- Limita los espacios a los tipos utilizados por las unidades.
    CONSTRAINT ck_espacio_hospitalario_tipo
        CHECK (tipo IN ('Clinica', 'Camilla', 'Quirofano', 'Sala', 'Cama')),
    -- Controla la disponibilidad operativa del espacio.
    CONSTRAINT ck_espacio_hospitalario_estado
        CHECK (estado IN ('Disponible', 'Ocupado', 'Mantenimiento', 'Inactivo')),
    -- Impide costos diarios negativos y permite NULL cuando no aplica.
    CONSTRAINT ck_espacio_hospitalario_costo_diario
        CHECK (costo_diario IS NULL OR costo_diario >= 0),
    -- Valida que el codigo del espacio no este vacio.
    CONSTRAINT ck_espacio_hospitalario_codigo_no_vacio
        CHECK (BTRIM(codigo) <> ''),
    CONSTRAINT fk_espacio_hospitalario_hospital
        FOREIGN KEY (hospital_id)
        REFERENCES hospital (hospital_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT fk_espacio_hospitalario_unidad
        FOREIGN KEY (unidad_medica_id)
        REFERENCES unidad_medica (unidad_medica_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL
);

```

```

-- Registra establecimientos externos utilizados en referencias y traslados.
CREATE TABLE IF NOT EXISTS institucion_externa (
    institucion_externa_id BIGINT GENERATED ALWAYS AS IDENTITY,
    direccion_id BIGINT,
    nombre VARCHAR(150) NOT NULL,
    tipo VARCHAR(30) NOT NULL,
    telefono VARCHAR(20),
    CONSTRAINT pk_institucion_externa PRIMARY KEY (institucion_externa_id),
    CONSTRAINT uq_institucion_externa_nombre UNIQUE (nombre),
    -- Clasifica las instituciones externas relacionadas con el hospital.
    CONSTRAINT ck_institucion_externa_tipo
        CHECK (tipo IN ('Hospital', 'Clinica', 'Laboratorio', 'Organizacion', 'Otra')),
    -- Valida que el nombre de la institucion no este vacio.
    CONSTRAINT ck_institucion_externa_nombre_no_vacio
        CHECK (BTRIM(nombre) <> ''),
    -- Valida el telefono opcional de la institucion.
    CONSTRAINT ck_institucion_externa_telefono_formato
        CHECK (telefono IS NULL OR telefono ~ '^[+>() 0-9]{8,20}$'),
    CONSTRAINT fk_institucion_externa_direccion
        FOREIGN KEY (direccion_id)
        REFERENCES direccion (direccion_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL
);

```

```

-- 2. PERSONAS, PACIENTES Y PERSONAL

-- Centraliza la informacion comun de pacientes, encargados y trabajadores.

CREATE TABLE IF NOT EXISTS persona (

    persona_id BIGINT GENERATED ALWAYS AS IDENTITY,

    direccion_id BIGINT,

    nombres VARCHAR(100) NOT NULL,

    apellidos VARCHAR(100) NOT NULL,

    dpi CHAR(13),

    fecha_nacimiento DATE NOT NULL,

    sexo VARCHAR(10) NOT NULL,

    estado_civil VARCHAR(15),

    telefono VARCHAR(20),

    correo VARCHAR(120),

    CONSTRAINT pk_persona PRIMARY KEY (persona_id),

    CONSTRAINT uq_persona_dpi UNIQUE (dpi),

    -- Valida que los nombres no sean una cadena vacia.

    CONSTRAINT ck_persona_nombres_no_vacios

        CHECK (BTRIM(nombres) <> ''),

    -- Valida que los apellidos no sean una cadena vacia.

    CONSTRAINT ck_persona_apellidos_no_vacios

        CHECK (BTRIM(apellidos) <> ''),

    -- Limita el sexo a los valores definidos en las fichas.

    CONSTRAINT ck_persona_sexo

        CHECK (sexo IN ('Masculino', 'Femenino')),

    -- Limita el estado civil a valores administrativos validos.

    CONSTRAINT ck_persona_estado_civil

        CHECK (estado_civil IS NULL OR estado_civil IN ('Soltero', 'Casado', 'Unido', 'Divorciado', 'Viudo')),

    -- Evita fechas de nacimiento anteriores al rango operativo del sistema.

    CONSTRAINT ck_persona_fecha_nacimiento

        CHECK (fecha_nacimiento >= DATE '1900-01-01'),

    -- Exige exactamente trece digitos cuando se registra DPI.

    CONSTRAINT ck_persona_dpi_formato

        CHECK (dpi IS NULL OR dpi ~ '^[0-9]{13}$'),

    -- Valida el telefono opcional de la persona.

    CONSTRAINT ck_persona_telefono_formato

        CHECK (telefono IS NULL OR telefono ~ '^[-+() 0-9]{8,20}$'),

    -- Realiza una validacion basica del correo opcional.

    CONSTRAINT ck_persona_correo_formato

        CHECK (correo IS NULL OR correo ~* '^[-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$'),

    CONSTRAINT fk_persona_direccion

```

```

        FOREIGN KEY (direccion_id)
        REFERENCES direccion (direccion_id)
        ON UPDATE CASCADE
        ON DELETE SET NULL
    );

-- Amplia a persona con los datos administrativos propios de un paciente.
CREATE TABLE IF NOT EXISTS paciente (
    paciente_id BIGINT,
    numero_expediente VARCHAR(25) NOT NULL,
    numero_seguro_social VARCHAR(30),
    fecha_registro DATE NOT NULL DEFAULT CURRENT_DATE,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_paciente PRIMARY KEY (paciente_id),
    CONSTRAINT uq_paciente_expediente UNIQUE (numero_expediente),
    CONSTRAINT uq_paciente_seguro_social UNIQUE (numero_seguro_social),
    -- Valida que el numero de expediente no este vacio.
    CONSTRAINT ck_paciente_expediente_no_vacio
        CHECK (BTRIM(numero_expediente) <> ''),
    CONSTRAINT fk_paciente_persona
        FOREIGN KEY (paciente_id)
        REFERENCES persona (persona_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);

-- Relaciona cada paciente con uno o varios encargados autorizados.
CREATE TABLE IF NOT EXISTS encargado_paciente (
    encargado_paciente_id BIGINT GENERATED ALWAYS AS IDENTITY,
    paciente_id BIGINT NOT NULL,
    encargado_persona_id BIGINT NOT NULL,
    parentesco VARCHAR(40) NOT NULL,
    es_principal BOOLEAN NOT NULL DEFAULT FALSE,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_encargado_paciente PRIMARY KEY (encargado_paciente_id),
    CONSTRAINT uq_encargado_paciente_relacion
        UNIQUE (paciente_id, encargado_persona_id),
    -- Valida que el parentesco no este vacio.
    CONSTRAINT ck_encargado_paciente_parentesco_no_vacio
        CHECK (BTRIM(parentesco) <> ''),

```

```

-- Impide que una persona sea registrada como su propio encargado.
CONSTRAINT ck_encargado_paciente_personas_distintas
    CHECK (paciente_id <> encargado_persona_id),
CONSTRAINT fk_encargado_paciente_paciente
    FOREIGN KEY (paciente_id)
    REFERENCES paciente (paciente_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
CONSTRAINT fk_encargado_paciente_persona
    FOREIGN KEY (encargado_persona_id)
    REFERENCES persona (persona_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
);

-- Registra personal contratado directamente por un hospital.
CREATE TABLE IF NOT EXISTS empleado (
    empleado_id BIGINT GENERATED ALWAYS AS IDENTITY,
    persona_id BIGINT NOT NULL,
    hospital_id BIGINT NOT NULL,
    codigo_empleado VARCHAR(20) NOT NULL,
    tipo_empleado VARCHAR(25) NOT NULL,
    fecha_contratacion DATE NOT NULL,
    fecha_finalizacion DATE,
    estado_laboral VARCHAR(15) NOT NULL DEFAULT 'Activo',
    CONSTRAINT pk_empleado PRIMARY KEY (empleado_id),
    CONSTRAINT uq_empleado_hospital_codigo UNIQUE (hospital_id, codigo_empleado),
    CONSTRAINT uq_empleado_persona_hospital UNIQUE (persona_id, hospital_id),
    -- Limita los tipos de personal contratados por el hospital.
    CONSTRAINT ck_empleado_tipo
        CHECK (tipo_empleado IN ('Medico', 'Enfermero', 'Practicante', 'Recepcionista', 'Administrativo',
'Otro')),
    -- Controla el estado laboral del empleado.
    CONSTRAINT ck_empleado_estado_laboral
        CHECK (estado_laboral IN ('Activo', 'Suspendido', 'Finalizado')),
    -- Impide que la fecha de finalizacion sea anterior a la contratacion.
    CONSTRAINT ck_empleado_fechas
        CHECK (fecha_finalizacion IS NULL OR fecha_finalizacion >= fecha_contratacion),
    -- Valida que el codigo de empleado no este vacio.
    CONSTRAINT ck_empleado_codigo_no_vacio

```

```

        CHECK (BTRIM(codigo_empleado) <> ''),
CONSTRAINT fk_empleado_persona
    FOREIGN KEY (persona_id)
    REFERENCES persona (persona_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
CONSTRAINT fk_empleado_hospital
    FOREIGN KEY (hospital_id)
    REFERENCES hospital (hospital_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
);

-- Cataloga especialidades medicas de consulta externa, emergencia y cirugia.
CREATE TABLE IF NOT EXISTS especialidad (
    especialidad_id SMALLINT GENERATED ALWAYS AS IDENTITY,
    nombre VARCHAR(120) NOT NULL,
    descripcion VARCHAR(300),
    activa BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_especialidad PRIMARY KEY (especialidad_id),
    CONSTRAINT uq_especialidad_nombre UNIQUE (nombre),
    -- Valida que el nombre de la especialidad no este vacio.
    CONSTRAINT ck_especialidad_nombre_no_vacio
        CHECK (BTRIM(nombre) <> '')
);

-- Registra los datos profesionales de medicos internos, residentes o externos.
CREATE TABLE IF NOT EXISTS medico (
    medico_id BIGINT,
    numero_colegiado VARCHAR(25) NOT NULL,
    clase_medico VARCHAR(12) NOT NULL,
    institucion_externa_id BIGINT,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_medico PRIMARY KEY (medico_id),
    CONSTRAINT uq_medico_numero_colegiado UNIQUE (numero_colegiado),
    -- Limita la clase profesional a las indicadas para cirugia.
    CONSTRAINT ck_medico_clase
        CHECK (clase_medico IN ('Residente', 'Interno', 'Externo')),
    -- Exige institucion externa solo para el medico clasificado como externo.
    CONSTRAINT ck_medico_institucion_externa

```

```

        CHECK (
            (clase_medico = 'Externo' AND institucion_externa_id IS NOT NULL)
            OR (clase_medico IN ('Residente', 'Interno') AND institucion_externa_id IS NULL)
        ),
-- Valida que el numero de colegiado no este vacio.
CONSTRAINT ck_medico_colegiado_no_vacio
    CHECK (BTRIM(numero_colegiado) <> ''),
CONSTRAINT fk_medico_persona
    FOREIGN KEY (medico_id)
    REFERENCES persona (persona_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,
CONSTRAINT fk_medico_institucion_externa
    FOREIGN KEY (institucion_externa_id)
    REFERENCES institucion_externa (institucion_externa_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
);

-- Resuelve la relacion muchos a muchos entre medicos y especialidades.
CREATE TABLE IF NOT EXISTS medico_especialidad (
    medico_id BIGINT NOT NULL,
    especialidad_id SMALLINT NOT NULL,
    es_principal BOOLEAN NOT NULL DEFAULT FALSE,
    CONSTRAINT pk_medico_especialidad
        PRIMARY KEY (medico_id, especialidad_id),
    CONSTRAINT fk_medico_especialidad_medico
        FOREIGN KEY (medico_id)
        REFERENCES medico (medico_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT fk_medico_especialidad_especialidad
        FOREIGN KEY (especialidad_id)
        REFERENCES especialidad (especialidad_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);

```

```

-- Registra la vinculacion de un medico con uno o varios hospitales.
CREATE TABLE IF NOT EXISTS medico_hospital (
    medico_hospital_id BIGINT GENERATED ALWAYS AS IDENTITY,
    medico_id BIGINT NOT NULL,
    hospital_id BIGINT NOT NULL,
    condicion VARCHAR(12) NOT NULL,
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_medico_hospital PRIMARY KEY (medico_hospital_id),
    CONSTRAINT uq_medico_hospital_vinculo UNIQUE (medico_id, hospital_id, fecha_inicio),
    -- Limita la condicion del medico en el hospital.
    CONSTRAINT ck_medico_hospital_condicion
        CHECK (condicion IN ('Residente', 'Interno', 'Externo')),
    -- Impide que el fin de la vinculacion sea anterior al inicio.
    CONSTRAINT ck_medico_hospital_fechas
        CHECK (fecha_fin IS NULL OR fecha_fin >= fecha_inicio),
    CONSTRAINT fk_medico_hospital_medico
        FOREIGN KEY (medico_id)
        REFERENCES medico (medico_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
    CONSTRAINT fk_medico_hospital_hospital
        FOREIGN KEY (hospital_id)
        REFERENCES hospital (hospital_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);

```

```

-- Especializa a un empleado con los datos profesionales de enfermeria.
CREATE TABLE IF NOT EXISTS enfermero (
    enfermero_id BIGINT,
    numero_registro VARCHAR(25),
    nivel VARCHAR(12) NOT NULL,
    CONSTRAINT pk_enfermero PRIMARY KEY (enfermero_id),
    CONSTRAINT uq_enfermero_numero_registro UNIQUE (numero_registro),
    -- Diferencia al enfermero registrado del practicante de enfermeria.
    CONSTRAINT ck_enfermero_nivel
        CHECK (nivel IN ('Registrado', 'Practicante')),
    -- Exige numero de registro al personal de enfermeria registrado.

```

```

CONSTRAINT ck_enfermero_registro_segun_nivel
    CHECK (
        (nivel = 'Registrado' AND numero_registro IS NOT NULL AND BTRIM(numero_registro) <> '')
        OR nivel = 'Practicante'
    ),
CONSTRAINT fk_enfermero_empleado
    FOREIGN KEY (enfermero_id)
    REFERENCES empleado (empleado_id)
    ON UPDATE CASCADE
    ON DELETE RESTRICT
);

```

-- Especializa a un empleado que realiza practica profesional de medicina.

```

CREATE TABLE IF NOT EXISTS practicante_medicina (
    practicante_id BIGINT,
    institucion_educativa VARCHAR(150) NOT NULL,
    numero_carnet VARCHAR(30) NOT NULL,
    fecha_inicio DATE NOT NULL,
    fecha_fin DATE NOT NULL,
    CONSTRAINT pk_practicante_medicina PRIMARY KEY (practicante_id),
    CONSTRAINT uq_practicante_medicina_carnet UNIQUE (institucion_educativa, numero_carnet),
    -- Valida que la institucion educativa no este vacia.
    CONSTRAINT ck_practicante_institucion_no_vacia
        CHECK (BTRIM(institucion_educativa) <> ''),
    -- Valida que el numero de carnet no este vacio.
    CONSTRAINT ck_practicante_carnet_no_vacio
        CHECK (BTRIM(numero_carnet) <> ''),
    -- Impide periodos de practica con fechas invertidas.
    CONSTRAINT ck_practicante_fechas
        CHECK (fecha_fin >= fecha_inicio),
    CONSTRAINT fk_practicante_medicina_empleado
        FOREIGN KEY (practicante_id)
        REFERENCES empleado (empleado_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);

```

-- Registra los turnos asignados al personal en una unidad medica.

```
CREATE TABLE IF NOT EXISTS turno_personal (  
    turno_personal_id BIGINT GENERATED ALWAYS AS IDENTITY,  
    empleado_id BIGINT NOT NULL,  
    unidad_medica_id BIGINT NOT NULL,  
    fecha DATE NOT NULL,  
    hora_inicio TIME NOT NULL,  
    hora_fin TIME NOT NULL,  
    estado VARCHAR(12) NOT NULL DEFAULT 'Programado',  
    CONSTRAINT pk_turno_personal PRIMARY KEY (turno_personal_id),  
    CONSTRAINT uq_turno_personal_asignacion  
        UNIQUE (empleado_id, unidad_medica_id, fecha, hora_inicio),  
    -- Impide turnos sin duracion o con hora final anterior.  
    CONSTRAINT ck_turno_personal_horas  
        CHECK (hora_fin > hora_inicio),  
    -- Controla el estado administrativo del turno.  
    CONSTRAINT ck_turno_personal_estado  
        CHECK (estado IN ('Programado', 'Cumplido', 'Cancelado')),  
    CONSTRAINT fk_turno_personal_empleado  
        FOREIGN KEY (empleado_id)  
        REFERENCES empleado (empleado_id)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT,  
    CONSTRAINT fk_turno_personal_unidad  
        FOREIGN KEY (unidad_medica_id)  
        REFERENCES unidad_medica (unidad_medica_id)  
        ON UPDATE CASCADE  
        ON DELETE RESTRICT  
);
```

-- Solo puede existir un encargado principal activo por paciente.

```
CREATE UNIQUE INDEX IF NOT EXISTS uq_encargado_principal_activo  
    ON encargado_paciente (paciente_id)  
    WHERE es_principal = TRUE AND activo = TRUE;
```

-- Solo puede existir una especialidad principal por medico.

```
CREATE UNIQUE INDEX IF NOT EXISTS uq_medico_especialidad_principal  
    ON medico_especialidad (medico_id)  
    WHERE es_principal = TRUE;
```

```
COMMIT;
```

```
/*
```

Verificacion rapida despues de ejecutar el archivo:

```
SELECT table_name
```

```
FROM information_schema.tables
```

```
WHERE table_schema = 'hospital'
```

```
ORDER BY table_name;
```

Resultado esperado en este bloque: 20 tablas.

```
*/
```

nucleo_clinico.sql

```
BEGIN;

SET search_path TO hospital, public;

-- Estas claves compuestas permiten comprobar el hospital de la unidad y el espacio.

ALTER TABLE unidad_medica

    ADD CONSTRAINT uq_unidad_medica_id_hospital UNIQUE (unidad_medica_id, hospital_id);

ALTER TABLE espacio_hospitalario

    ADD CONSTRAINT uq_espacio_id_hospital UNIQUE (espacio_hospitalario_id, hospital_id);

-- Agrupa las atenciones de un paciente en un hospital durante un episodio.

CREATE TABLE episodio_atencion (

    episodio_atencion_id BIGINT GENERATED ALWAYS AS IDENTITY,

    paciente_id BIGINT NOT NULL,

    hospital_id BIGINT NOT NULL,

    fecha_apertura TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    fecha_cierre TIMESTAMPTZ,

    estado VARCHAR(12) NOT NULL DEFAULT 'Abierto',

    motivo_apertura TEXT NOT NULL,

    CONSTRAINT pk_episodio_atencion PRIMARY KEY (episodio_atencion_id),

    CONSTRAINT uq_episodio_hospital UNIQUE (episodio_atencion_id, hospital_id),

    -- Solo se permiten los estados del ciclo de vida del episodio.

    CONSTRAINT ck_episodio_estado CHECK (estado IN ('Abierto', 'Cerrado')),

    -- El cierre debe ser posterior a la apertura.

    CONSTRAINT ck_episodio_fechas CHECK (fecha_cierre IS NULL OR fecha_cierre >= fecha_apertura),

    -- Un episodio cerrado requiere fecha de cierre.

    CONSTRAINT ck_episodio_cierre CHECK (

        (estado = 'Abierto' AND fecha_cierre IS NULL)

        OR (estado = 'Cerrado' AND fecha_cierre IS NOT NULL)

    ),

    -- El motivo de apertura debe contener texto.

    CONSTRAINT ck_episodio_motivo CHECK (BTRIM(motivo_apertura) <> ''),

    CONSTRAINT fk_episodio_paciente FOREIGN KEY (paciente_id)

        REFERENCES paciente (paciente_id) ON DELETE RESTRICT ON UPDATE CASCADE,

    CONSTRAINT fk_episodio_hospital FOREIGN KEY (hospital_id)

        REFERENCES hospital (hospital_id) ON DELETE RESTRICT ON UPDATE CASCADE

);
```

-- Registra el ingreso de un paciente a emergencias, cirugía u hospitalización.

```
CREATE TABLE ingreso (  
    ingreso_id BIGINT GENERATED ALWAYS AS IDENTITY,  
    episodio_atencion_id BIGINT NOT NULL,  
    hospital_id BIGINT NOT NULL,  
    unidad_medica_id BIGINT NOT NULL,  
    servicio_medico_id SMALLINT NOT NULL,  
    medico_encargado_id BIGINT NOT NULL,  
    encargado_paciente_id BIGINT,  
    espacio_hospitalario_id BIGINT,  
    fecha_ingreso TIMESTAMPTZ NOT NULL,  
    motivo_ingreso TEXT NOT NULL,  
    diagnostico_presuntivo TEXT NOT NULL,  
    estado VARCHAR(10) NOT NULL DEFAULT 'Abierto',  
    CONSTRAINT pk_ingreso PRIMARY KEY (ingreso_id),  
    CONSTRAINT uq_ingreso_episodio UNIQUE (ingreso_id, episodio_atencion_id),  
    -- El ingreso permanece abierto hasta que se documenta su egreso.  
    CONSTRAINT ck_ingreso_estado CHECK (estado IN ('Abierto', 'Cerrado')),  
    -- Exige un motivo de ingreso descriptivo.  
    CONSTRAINT ck_ingreso_motivo CHECK (BTRIM(motivo_ingreso) <> ''),  
    -- Exige un diagnostico presuntivo.  
    CONSTRAINT ck_ingreso_diagnostico CHECK (BTRIM(diagnostico_presuntivo) <> ''),  
    CONSTRAINT fk_ingreso_episodio_hospital  
        FOREIGN KEY (episodio_atencion_id, hospital_id)  
        REFERENCES episodio_atencion (episodio_atencion_id, hospital_id)  
        ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_ingreso_unidad_hospital  
        FOREIGN KEY (unidad_medica_id, hospital_id)  
        REFERENCES unidad_medica (unidad_medica_id, hospital_id)  
        ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_ingreso_unidad_servicio  
        FOREIGN KEY (unidad_medica_id, servicio_medico_id)  
        REFERENCES unidad_servicio (unidad_medica_id, servicio_medico_id)  
        ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_ingreso_medico FOREIGN KEY (medico_encargado_id)  
        REFERENCES medico (medico_id) ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_ingreso_encargado FOREIGN KEY (encargado_paciente_id)  
        REFERENCES encargado_paciente (encargado_paciente_id)  
        ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_ingreso_espacio_hospital
```

```

        FOREIGN KEY (espacio_hospitalario_id, hospital_id)
        REFERENCES espacio_hospitalario (espacio_hospitalario_id, hospital_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
    );

-- Cataloga diagnosticos clinicos que pueden documentarse en el egreso.
CREATE TABLE diagnostico (
    diagnostico_id BIGINT GENERATED ALWAYS AS IDENTITY,
    codigo VARCHAR(25),
    nombre VARCHAR(200) NOT NULL,
    descripcion TEXT,
    CONSTRAINT pk_diagnostico PRIMARY KEY (diagnostico_id),
    CONSTRAINT uq_diagnostico_codigo UNIQUE (codigo),
    -- Impide diagnosticos sin nombre.
    CONSTRAINT ck_diagnostico_nombre CHECK (BTRIM(nombre) <> '')
);

-- Documenta el egreso de la unidad y su destino o motivo de salida.
CREATE TABLE egreso (
    egreso_id BIGINT GENERATED ALWAYS AS IDENTITY,
    ingreso_id BIGINT NOT NULL,
    medico_asignado_id BIGINT NOT NULL,
    institucion_referida_id BIGINT,
    fecha_egreso TIMESTAMPTZ NOT NULL,
    motivo_egreso TEXT NOT NULL,
    codigo_egreso VARCHAR(10) NOT NULL,
    sin_consentimiento_medico BOOLEAN NOT NULL DEFAULT FALSE,
    motivo_sin_consentimiento TEXT,
    operaciones_intervenciones TEXT,
    codigo_traslado VARCHAR(40),
    observaciones TEXT,
    CONSTRAINT pk_egreso PRIMARY KEY (egreso_id),
    CONSTRAINT uq_egreso_ingreso UNIQUE (ingreso_id),
    -- Limita el codigo de egreso a los cuatro valores de la ficha.
    CONSTRAINT ck_egreso_codigo CHECK (codigo_egreso IN ('Vivo', 'Muerto', 'Embarazo', 'Parto')),
    -- Un egreso sin consentimiento medico exige explicacion.
    CONSTRAINT ck_egreso_sin_consentimiento CHECK (
        (sin_consentimiento_medico AND NULLIF(BTRIM(motivo_sin_consentimiento), '') IS NOT NULL)
        OR (NOT sin_consentimiento_medico AND motivo_sin_consentimiento IS NULL)
    ),
);

```

```

-- El motivo del egreso debe estar documentado.
CONSTRAINT ck_egreso_motivo CHECK (BTRIM(motivo_egreso) <> ''),
CONSTRAINT fk_egreso_ingreso FOREIGN KEY (ingreso_id)
    REFERENCES ingreso (ingreso_id) ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_egreso_medico FOREIGN KEY (medico_asignado_id)
    REFERENCES medico (medico_id) ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_egreso_institucion FOREIGN KEY (institucion_referida_id)
    REFERENCES institucion_externa (institucion_externa_id)
    ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Asocia diagnosticos principales y secundarios a un egreso.
CREATE TABLE egreso_diagnostico (
    egreso_id BIGINT NOT NULL,
    diagnostico_id BIGINT NOT NULL,
    tipo VARCHAR(10) NOT NULL,
    observaciones TEXT,
    CONSTRAINT pk_egreso_diagnostico PRIMARY KEY (egreso_id, diagnostico_id),
    -- Distingue el diagnostico principal de los secundarios.
    CONSTRAINT ck_egreso_diagnostico_tipo CHECK (tipo IN ('Principal', 'Secundario')),
    CONSTRAINT fk_egreso_diagnostico_egreso FOREIGN KEY (egreso_id)
        REFERENCES egreso (egreso_id) ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_egreso_diagnostico_diagnostico FOREIGN KEY (diagnostico_id)
        REFERENCES diagnostico (diagnostico_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Registra un traslado entre unidades de la cadena o hacia otra institucion.
CREATE TABLE traslado (
    traslado_id BIGINT GENERATED ALWAYS AS IDENTITY,
    episodio_atencion_id BIGINT NOT NULL,
    ingreso_origen_id BIGINT,
    medico_indicador_id BIGINT NOT NULL,
    unidad_origen_id BIGINT NOT NULL,
    unidad_destino_id BIGINT,
    institucion_destino_id BIGINT,
    servicio_destino_id SMALLINT,
    fecha_traslado TIMESTAMPTZ NOT NULL,
    tipo VARCHAR(10) NOT NULL,
    motivo TEXT NOT NULL,
    codigo_traslado VARCHAR(40) NOT NULL,

```

```

CONSTRAINT pk_traslado PRIMARY KEY (traslado_id),
CONSTRAINT uq_traslado_codigo UNIQUE (codigo_traslado),
-- Diferencia un traslado interno de uno externo.
CONSTRAINT ck_traslado_tipo CHECK (tipo IN ('Interno', 'Externo')),
-- Interno requiere unidad destino; externo requiere institucion externa.
CONSTRAINT ck_traslado_destino CHECK (
    (tipo = 'Interno' AND unidad_destino_id IS NOT NULL AND institucion_destino_id IS NULL)
    OR (tipo = 'Externo' AND unidad_destino_id IS NULL AND institucion_destino_id IS NOT NULL)
),
-- Impide indicar la misma unidad como origen y destino internos.
CONSTRAINT ck_traslado_unidades_distintas CHECK (
    unidad_destino_id IS NULL OR unidad_origen_id <> unidad_destino_id
),
-- El motivo del traslado debe estar documentado.
CONSTRAINT ck_traslado_motivo CHECK (BTRIM(motivo) <> ''),
CONSTRAINT fk_traslado_episodio FOREIGN KEY (episodio_atencion_id)
    REFERENCES episodio_atencion (episodio_atencion_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_traslado_ingreso_episodio
    FOREIGN KEY (ingreso_origen_id, episodio_atencion_id)
    REFERENCES ingreso (ingreso_id, episodio_atencion_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_traslado_medico FOREIGN KEY (medico_indicador_id)
    REFERENCES medico (medico_id) ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_traslado_origen FOREIGN KEY (unidad_origen_id)
    REFERENCES unidad_medica (unidad_medica_id) ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_traslado_destino FOREIGN KEY (unidad_destino_id)
    REFERENCES unidad_medica (unidad_medica_id) ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_traslado_institucion FOREIGN KEY (institucion_destino_id)
    REFERENCES institucion_externa (institucion_externa_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_traslado_servicio FOREIGN KEY (servicio_destino_id)
    REFERENCES servicio_medico (servicio_medico_id)
    ON DELETE RESTRICT ON UPDATE CASCADE
);

```

```

-- Conserva el consentimiento para traslado, cirugia u hospitalizacion.
CREATE TABLE consentimiento_general (
    consentimiento_general_id BIGINT GENERATED ALWAYS AS IDENTITY,
    episodio_atencion_id BIGINT NOT NULL,
    firmante_persona_id BIGINT NOT NULL,
    medico_solicitante_id BIGINT NOT NULL,
    tipo VARCHAR(17) NOT NULL,
    fecha TIMESTAMPTZ NOT NULL,
    aceptado BOOLEAN NOT NULL,
    descripcion_procedimiento TEXT NOT NULL,
    observaciones TEXT,
    CONSTRAINT pk_consentimiento_general PRIMARY KEY (consentimiento_general_id),
    -- Limita el consentimiento a los procedimientos generales indicados.
    CONSTRAINT ck_consentimiento_general_tipo
        CHECK (tipo IN ('Traslado', 'Cirugia', 'Hospitalizacion')),
    -- Exige una descripcion clara del procedimiento aceptado o rechazado.
    CONSTRAINT ck_consentimiento_general_descripcion
        CHECK (BTRIM(descripcion_procedimiento) <> ''),
    CONSTRAINT fk_consentimiento_episodio FOREIGN KEY (episodio_atencion_id)
        REFERENCES episodio_atencion (episodio_atencion_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_consentimiento_firmante FOREIGN KEY (firmante_persona_id)
        REFERENCES persona (persona_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_consentimiento_medico FOREIGN KEY (medico_solicitante_id)
        REFERENCES medico (medico_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Como maximo puede existir un diagnostico principal por egreso.
CREATE UNIQUE INDEX uq_egreso_un_diagnostico_principal
    ON egreso_diagnostico (egreso_id)
    WHERE tipo = 'Principal';

COMMIT;

/*
Verificacion:
SELECT count(*) FROM information_schema.tables
WHERE table_schema = 'hospital' AND table_type = 'BASE TABLE';

Resultado acumulado esperado: 27 tablas.
*/

```

consulta_emergencia.sql

```
/*
Proyecto 1 - Bloque 3: consulta externa y emergencias.
Ejecutar despues del bloque de nucleo clinico.
*/
BEGIN;
SET search_path TO hospital, public;

-- Registra los horarios matutinos de consulta de los medicos por clinica.
CREATE TABLE horario_consulta (
    horario_consulta_id BIGINT GENERATED ALWAYS AS IDENTITY,
    medico_id BIGINT NOT NULL,
    unidad_medica_id BIGINT NOT NULL,
    clinica_id BIGINT NOT NULL,
    dia_semana SMALLINT NOT NULL,
    hora_inicio TIME NOT NULL,
    hora_fin TIME NOT NULL,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_horario_consulta PRIMARY KEY (horario_consulta_id),
    CONSTRAINT uq_horario_consulta UNIQUE (medico_id, clinica_id, dia_semana, hora_inicio),
    -- Usa ISO: lunes 1 hasta domingo 7.
    CONSTRAINT ck_horario_dia CHECK (dia_semana BETWEEN 1 AND 7),
    -- La consulta debe empezar y terminar en horario matutino.
    CONSTRAINT ck_horario_matutino CHECK (hora_inicio >= TIME '06:00'
        AND hora_fin <= TIME '12:00' AND hora_fin > hora_inicio),
    CONSTRAINT fk_horario_medico FOREIGN KEY (medico_id)
        REFERENCES medico (medico_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_horario_unidad FOREIGN KEY (unidad_medica_id)
        REFERENCES unidad_medica (unidad_medica_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_horario_clinica FOREIGN KEY (clinica_id)
        REFERENCES espacio_hospitalario (espacio_hospitalario_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Registra citas programadas, realizadas, reprogramadas y canceladas.
CREATE TABLE cita (
    cita_id BIGINT GENERATED ALWAYS AS IDENTITY,
    paciente_id BIGINT NOT NULL,
    medico_id BIGINT NOT NULL,
    unidad_medica_id BIGINT NOT NULL,
```

```

clinica_id BIGINT NOT NULL,

cita_anterior_id BIGINT,

fecha_hora TIMESTAMPTZ NOT NULL,

canal VARCHAR(10) NOT NULL,

tipo VARCHAR(12) NOT NULL,

estado VARCHAR(15) NOT NULL DEFAULT 'Programada',

precio_base NUMERIC(12,2) NOT NULL,

aplica_recargo_cancelacion BOOLEAN NOT NULL DEFAULT FALSE,

motivo_cancelacion TEXT,

fecha_registro TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT pk_cita PRIMARY KEY (cita_id),

CONSTRAINT uq_cita_medico_hora UNIQUE (medico_id, fecha_hora),

CONSTRAINT uq_cita_clinica_hora UNIQUE (clinica_id, fecha_hora),

-- Canal usado para solicitar la cita.

CONSTRAINT ck_cita_canal CHECK (canal IN ('Telefono', 'Correo', 'Recepcion')),

-- Primera consulta, reconsulta o referencia institucional.

CONSTRAINT ck_cita_tipo CHECK (tipo IN ('Primera', 'Reconsulta', 'Referida')),

-- Estado administrativo de la cita.

CONSTRAINT ck_cita_estado CHECK (estado IN ('Programada', 'Realizada', 'Reprogramada', 'Cancelada')),

-- El precio base de la consulta no puede ser negativo.

CONSTRAINT ck_cita_precio CHECK (precio_base >= 0),

-- Una cita cancelada debe indicar su motivo.

CONSTRAINT ck_cita_cancelacion CHECK (

    estado <> 'Cancelada' OR NULLIF(BTRIM(motivo_cancelacion), '') IS NOT NULL

),

-- Una cita no puede ser su propia cita anterior.

CONSTRAINT ck_cita_anterior_distinta CHECK (cita_anterior_id IS NULL OR cita_anterior_id <> cita_id),

CONSTRAINT fk_cita_paciente FOREIGN KEY (paciente_id)

    REFERENCES paciente (paciente_id) ON DELETE RESTRICT ON UPDATE CASCADE,

CONSTRAINT fk_cita_medico FOREIGN KEY (medico_id)

    REFERENCES medico (medico_id) ON DELETE RESTRICT ON UPDATE CASCADE,

CONSTRAINT fk_cita_unidad FOREIGN KEY (unidad_medica_id)

    REFERENCES unidad_medica (unidad_medica_id) ON DELETE RESTRICT ON UPDATE CASCADE,

CONSTRAINT fk_cita_clinica FOREIGN KEY (clinica_id)

    REFERENCES espacio_hospitalario (espacio_hospitalario_id)

    ON DELETE RESTRICT ON UPDATE CASCADE,

CONSTRAINT fk_cita_anterior FOREIGN KEY (cita_anterior_id)

    REFERENCES cita (cita_id) ON DELETE RESTRICT ON UPDATE CASCADE

);

```

```

-- Conserva el historial de transiciones y sus motivos para cada cita.
CREATE TABLE historial_estado_cita (
    historial_estado_cita_id BIGINT GENERATED ALWAYS AS IDENTITY,
    cita_id BIGINT NOT NULL,
    estado_anterior VARCHAR(15),
    estado_nuevo VARCHAR(15) NOT NULL,
    fecha_cambio TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    motivo TEXT,
    CONSTRAINT pk_historial_estado_cita PRIMARY KEY (historial_estado_cita_id),
    -- Limita el estado anterior a los valores posibles cuando existe.
    CONSTRAINT ck_historial_estado_anterior CHECK (estado_anterior IS NULL OR
        estado_anterior IN ('Programada', 'Realizada', 'Reprogramada', 'Cancelada')),
    -- Limita el nuevo estado a los valores posibles.
    CONSTRAINT ck_historial_estado_nuevo CHECK (estado_nuevo IN
        ('Programada', 'Realizada', 'Reprogramada', 'Cancelada')),
    CONSTRAINT fk_historial_cita FOREIGN KEY (cita_id)
        REFERENCES cita (cita_id) ON DELETE CASCADE ON UPDATE CASCADE
);

```

```

-- Documenta la consulta realizada, su diagnostico y observaciones.
CREATE TABLE consulta_externa (
    consulta_externa_id BIGINT GENERATED ALWAYS AS IDENTITY,
    cita_id BIGINT NOT NULL,
    fecha_atencion TIMESTAMPTZ NOT NULL,
    diagnostico TEXT NOT NULL,
    datos_interes TEXT,
    observaciones TEXT,
    orientacion_paciente TEXT,
    CONSTRAINT pk_consulta_externa PRIMARY KEY (consulta_externa_id),
    CONSTRAINT uq_consulta_externa_cita UNIQUE (cita_id),
    -- Exige documentar el diagnostico de la consulta.
    CONSTRAINT ck_consulta_diagnostico CHECK (BTRIM(diagnostico) <> ''),
    CONSTRAINT fk_consulta_cita FOREIGN KEY (cita_id)
        REFERENCES cita (cita_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

```

-- Registra recetas emitidas durante una consulta.

```
CREATE TABLE receta (  
    receta_id BIGINT GENERATED ALWAYS AS IDENTITY,  
    consulta_externa_id BIGINT NOT NULL,  
    fecha_emision DATE NOT NULL,  
    fecha_proxima_cita DATE,  
    indicaciones TEXT,  
    CONSTRAINT pk_receta PRIMARY KEY (receta_id),  
    -- La fecha de seguimiento no debe preceder la emision.  
    CONSTRAINT ck_receta_proxima_cita CHECK (  
        fecha_proxima_cita IS NULL OR fecha_proxima_cita >= fecha_emision  
    ),  
    CONSTRAINT fk_receta_consulta FOREIGN KEY (consulta_externa_id)  
        REFERENCES consulta_externa (consulta_externa_id)  
        ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

-- Cataloga medicamentos con su presentacion farmacologica.

```
CREATE TABLE medicamento (  
    medicamento_id BIGINT GENERATED ALWAYS AS IDENTITY,  
    nombre VARCHAR(120) NOT NULL,  
    presentacion VARCHAR(100) NOT NULL,  
    activo BOOLEAN NOT NULL DEFAULT TRUE,  
    CONSTRAINT pk_medicamento PRIMARY KEY (medicamento_id),  
    CONSTRAINT uq_medicamento_nombre_presentacion UNIQUE (nombre, presentacion),  
    -- Evita medicamentos sin nombre.  
    CONSTRAINT ck_medicamento_nombre CHECK (BTRIM(nombre) <> ''),  
    -- Exige una presentacion.  
    CONSTRAINT ck_medicamento_presentacion CHECK (BTRIM(presentacion) <> '')  
);
```

-- Detalla el medicamento, dosis y duracion indicada en una receta.

```
CREATE TABLE receta_detalle (  
    receta_detalle_id BIGINT GENERATED ALWAYS AS IDENTITY,  
    receta_id BIGINT NOT NULL,  
    medicamento_id BIGINT NOT NULL,  
    dosis VARCHAR(100) NOT NULL,  
    frecuencia VARCHAR(100) NOT NULL,  
    duracion_dias SMALLINT NOT NULL,  
    instrucciones TEXT,
```

```

CONSTRAINT pk_receta_detalle PRIMARY KEY (receta_detalle_id),
-- La duracion debe ser positiva.
CONSTRAINT ck_receta_detalle_duracion CHECK (duracion_dias > 0),
-- Exige indicar la dosis.
CONSTRAINT ck_receta_detalle_dosis CHECK (BTRIM(dosis) <> ''),
-- Exige indicar la frecuencia.
CONSTRAINT ck_receta_detalle_frecuencia CHECK (BTRIM(frecuencia) <> ''),
CONSTRAINT fk_receta_detalle_receta FOREIGN KEY (receta_id)
    REFERENCES receta (receta_id) ON DELETE CASCADE ON UPDATE CASCADE,
CONSTRAINT fk_receta_detalle_medimento FOREIGN KEY (medimento_id)
    REFERENCES medicamento (medimento_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

```

-- Registra ordenes de laboratorio solicitadas durante la consulta.

```

CREATE TABLE orden_laboratorio (
    orden_laboratorio_id BIGINT GENERATED ALWAYS AS IDENTITY,
    consulta_externa_id BIGINT NOT NULL,
    fecha_emision TIMESTAMPTZ NOT NULL,
    indicaciones TEXT,
    estado VARCHAR(12) NOT NULL DEFAULT 'Pendiente',
    CONSTRAINT pk_orden_laboratorio PRIMARY KEY (orden_laboratorio_id),
    -- Estado de seguimiento de la orden.
    CONSTRAINT ck_orden_laboratorio_estado CHECK (
        estado IN ('Pendiente', 'Realizada', 'Cancelada')
    ),
    CONSTRAINT fk_orden_consulta FOREIGN KEY (consulta_externa_id)
        REFERENCES consulta_externa (consulta_externa_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

```

-- Detalla los exámenes incluidos en una orden de laboratorio.

```

CREATE TABLE orden_laboratorio_detalle (
    orden_laboratorio_detalle_id BIGINT GENERATED ALWAYS AS IDENTITY,
    orden_laboratorio_id BIGINT NOT NULL,
    examen VARCHAR(150) NOT NULL,
    observaciones TEXT,
    CONSTRAINT pk_orden_laboratorio_detalle PRIMARY KEY (orden_laboratorio_detalle_id),
    -- Evita un nombre de examen vacío.
    CONSTRAINT ck_orden_detalle_examen CHECK (BTRIM(examen) <> ''),
    CONSTRAINT fk_orden_detalle_orden FOREIGN KEY (orden_laboratorio_id)

```

```

        REFERENCES orden_laboratorio (orden_laboratorio_id)

        ON DELETE CASCADE ON UPDATE CASCADE

    );

-- Registra la evaluacion, prioridad y resultado de un ingreso por emergencia.
CREATE TABLE atencion_emergencia (
    atencion_emergencia_id BIGINT GENERATED ALWAYS AS IDENTITY,
    ingreso_id BIGINT NOT NULL,
    fecha_evaluacion TIMESTAMPTZ NOT NULL,
    prioridad SMALLINT NOT NULL,
    estado_clinico TEXT NOT NULL,
    resultado TEXT,
    CONSTRAINT pk_atencion_emergencia PRIMARY KEY (atencion_emergencia_id),
    CONSTRAINT uq_atencion_emergencia_ingreso UNIQUE (ingreso_id),
    -- La prioridad de triaje va de uno a cinco; uno es la mayor urgencia.
    CONSTRAINT ck_emergencia_prioridad CHECK (prioridad BETWEEN 1 AND 5),
    -- Exige describir el estado clinico inicial.
    CONSTRAINT ck_emergencia_estado_clinico CHECK (BTRIM(estado_clinico) <> ''),
    CONSTRAINT fk_emergencia_ingreso FOREIGN KEY (ingreso_id)
        REFERENCES ingreso (ingreso_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Cataloga los procedimientos basicos y especializados de emergencias.
CREATE TABLE procedimiento_emergencia (
    procedimiento_emergencia_id BIGINT GENERATED ALWAYS AS IDENTITY,
    nombre VARCHAR(180) NOT NULL,
    tipo VARCHAR(15) NOT NULL,
    descripcion TEXT,
    costo_base NUMERIC(12,2) NOT NULL DEFAULT 0,
    CONSTRAINT pk_procedimiento_emergencia PRIMARY KEY (procedimiento_emergencia_id),
    CONSTRAINT uq_procedimiento_emergencia_nombre UNIQUE (nombre),
    -- Diferencia la atencion basica de la especializada.
    CONSTRAINT ck_procedimiento_emergencia_tipo CHECK (tipo IN ('Basico', 'Especializado')),
    -- El costo base no puede ser negativo.
    CONSTRAINT ck_procedimiento_emergencia_costo CHECK (costo_base >= 0),
    -- Exige un nombre para el procedimiento.
    CONSTRAINT ck_procedimiento_emergencia_nombre CHECK (BTRIM(nombre) <> '')
);

```

-- Documenta los procedimientos realizados a un paciente en emergencia.

```
CREATE TABLE emergencia_procedimiento (  
    emergencia_procedimiento_id BIGINT GENERATED ALWAYS AS IDENTITY,  
    atencion_emergencia_id BIGINT NOT NULL,  
    procedimiento_emergencia_id BIGINT NOT NULL,  
    medico_id BIGINT NOT NULL,  
    fecha_realizacion TIMESTAMPTZ NOT NULL,  
    resultado TEXT,  
    costo_aplicado NUMERIC(12,2) NOT NULL,  
    CONSTRAINT pk_emergencia_procedimiento PRIMARY KEY (emergencia_procedimiento_id),  
    -- El costo historico aplicado no puede ser negativo.  
    CONSTRAINT ck_emergencia_procedimiento_costo CHECK (costo_aplicado >= 0),  
    CONSTRAINT fk_emergencia_procedimiento_atencion FOREIGN KEY (atencion_emergencia_id)  
        REFERENCES atencion_emergencia (atencion_emergencia_id)  
        ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_emergencia_procedimiento_catalogo FOREIGN KEY (procedimiento_emergencia_id)  
        REFERENCES procedimiento_emergencia (procedimiento_emergencia_id)  
        ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_emergencia_procedimiento_medico FOREIGN KEY (medico_id)  
        REFERENCES medico (medico_id) ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

-- Registra la espera por camilla segun prioridad y orden de llegada.

```
CREATE TABLE cola_emergencia (  
    cola_emergencia_id BIGINT GENERATED ALWAYS AS IDENTITY,  
    atencion_emergencia_id BIGINT NOT NULL,  
    fecha_inicio TIMESTAMPTZ NOT NULL,  
    fecha_fin TIMESTAMPTZ,  
    prioridad SMALLINT NOT NULL,  
    estado VARCHAR(12) NOT NULL DEFAULT 'Esperando',  
    CONSTRAINT pk_cola_emergencia PRIMARY KEY (cola_emergencia_id),  
    -- La prioridad de espera va de uno a cinco.  
    CONSTRAINT ck_cola_prioridad CHECK (prioridad BETWEEN 1 AND 5),  
    -- Limita los estados de la cola de espera.  
    CONSTRAINT ck_cola_estado CHECK (estado IN ('Esperando', 'Asignado', 'Cancelado')),  
    -- El fin de espera debe ser posterior al inicio.  
    CONSTRAINT ck_cola_fechas CHECK (fecha_fin IS NULL OR fecha_fin >= fecha_inicio),  
    -- Solo la espera activa carece de fecha final.  
    CONSTRAINT ck_cola_cierre CHECK (  
        (estado = 'Esperando' AND fecha_fin IS NULL)
```

```

        OR (estado <> 'Esperando' AND fecha_fin IS NOT NULL)
    ),
    CONSTRAINT fkColaAtencion FOREIGN KEY (atencion_emergencia_id)
        REFERENCES atencion_emergencia (atencion_emergencia_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Un paciente no puede estar dos veces en la cola activa de una atencion.
CREATE UNIQUE INDEX uqColaEmergenciaActiva
    ON cola_emergencia (atencion_emergencia_id)
    WHERE estado = 'Esperando';

COMMIT;

/*
Verificacion: 27 tablas anteriores + 13 nuevas = 40 tablas.
SELECT COUNT(*) FROM information_schema.tables
WHERE table_schema = 'hospital' AND table_type = 'BASE TABLE';
*/

```

cirugia.sql

```
BEGIN;

SET search_path TO hospital, public;

-- Registra la solicitud de una cirugia urgente o programada.

CREATE TABLE solicitud_cirugia (

    solicitud_cirugia_id BIGINT GENERATED ALWAYS AS IDENTITY,

    paciente_id BIGINT NOT NULL,

    hospital_id BIGINT NOT NULL,

    cirujano_id BIGINT NOT NULL,

    episodio_atencion_id BIGINT,

    fecha_solicitud TIMESTAMPTZ NOT NULL,

    caracter VARCHAR(10) NOT NULL,

    historia_clinica TEXT NOT NULL,

    procedimiento_propuesto TEXT NOT NULL,

    duracion_estimada_minutos INTEGER NOT NULL,

    tipo_anestesia VARCHAR(80) NOT NULL,

    observaciones TEXT,

    CONSTRAINT pk_solicitud_cirugia PRIMARY KEY (solicitud_cirugia_id),

    CONSTRAINT uq_solicitud_paciente UNIQUE (solicitud_cirugia_id, paciente_id),

    -- Solo se permiten cirugias urgentes o programadas.

    CONSTRAINT ck_solicitud_caracter CHECK (caracter IN ('Urgente', 'Programado')),

    -- El tiempo estimado debe ser mayor que cero.

    CONSTRAINT ck_solicitud_duracion CHECK (duracion_estimada_minutos > 0),

    -- Exige la historia clinica necesaria para solicitar cirugia.

    CONSTRAINT ck_solicitud_historia CHECK (BTRIM(historia_clinica) <> ''),

    -- Exige describir el procedimiento solicitado.

    CONSTRAINT ck_solicitud_procedimiento CHECK (BTRIM(procedimiento_propuesto) <> ''),

    -- Exige el tipo de anestesia previsto.

    CONSTRAINT ck_solicitud_anestesia CHECK (BTRIM(tipo_anestesia) <> ''),

    CONSTRAINT fk_solicitud_paciente FOREIGN KEY (paciente_id)

        REFERENCES paciente (paciente_id) ON DELETE RESTRICT ON UPDATE CASCADE,

    CONSTRAINT fk_solicitud_hospital FOREIGN KEY (hospital_id)

        REFERENCES hospital (hospital_id) ON DELETE RESTRICT ON UPDATE CASCADE,

    CONSTRAINT fk_solicitud_cirujano FOREIGN KEY (cirujano_id)

        REFERENCES medico (medico_id) ON DELETE RESTRICT ON UPDATE CASCADE,

    CONSTRAINT fk_solicitud_episodio_hospital

        FOREIGN KEY (episodio_atencion_id, hospital_id)

        REFERENCES episodio_atencion (episodio_atencion_id, hospital_id)

        ON DELETE RESTRICT ON UPDATE CASCADE
```

```
);
```

```
-- Guarda la decision y razones del comite para una solicitud quirurgica.
```

```
CREATE TABLE evaluacion_solicitud_cirugia (  
    evaluacion_solicitud_cirugia_id BIGINT GENERATED ALWAYS AS IDENTITY,  
    solicitud_cirugia_id BIGINT NOT NULL,  
    medico_evaluador_id BIGINT NOT NULL,  
    fecha_evaluacion TIMESTAMPTZ NOT NULL,  
    decision VARCHAR(10) NOT NULL,  
    razones TEXT,  
    CONSTRAINT pk_evaluacion_solicitud PRIMARY KEY (evaluacion_solicitud_cirugia_id),  
    CONSTRAINT uq_evaluacion_solicitud UNIQUE (solicitud_cirugia_id),  
    CONSTRAINT uq_evaluacion_solicitud_decision  
        UNIQUE (evaluacion_solicitud_cirugia_id, solicitud_cirugia_id, decision),  
    -- Limita la decision del comite a aprobada o rechazada.  
    CONSTRAINT ck_evaluacion_decision CHECK (decision IN ('Aprobada', 'Rechazada')),  
    -- Una solicitud rechazada exige explicar las razones.  
    CONSTRAINT ck_evaluacion_razones CHECK (  
        decision <> 'Rechazada' OR NULLIF(BTRIM(razones), '') IS NOT NULL  
    ),  
    CONSTRAINT fk_evaluacion_solicitud FOREIGN KEY (solicitud_cirugia_id)  
        REFERENCES solicitud_cirugia (solicitud_cirugia_id)  
        ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_evaluacion_medico FOREIGN KEY (medico_evaluador_id)  
        REFERENCES medico (medico_id) ON DELETE RESTRICT ON UPDATE CASCADE  
);
```

```
-- Registra la cirugia aprobada, el quirofano y el horario asignado.
```

```
CREATE TABLE cirugia (  
    cirugia_id BIGINT GENERATED ALWAYS AS IDENTITY,  
    solicitud_cirugia_id BIGINT NOT NULL,  
    evaluacion_solicitud_cirugia_id BIGINT NOT NULL,  
    decision_requerida VARCHAR(10) NOT NULL DEFAULT 'Aprobada',  
    quirofano_id BIGINT NOT NULL,  
    fecha_programada TIMESTAMPTZ NOT NULL,  
    fecha_inicio TIMESTAMPTZ,  
    fecha_fin TIMESTAMPTZ,  
    estado VARCHAR(12) NOT NULL DEFAULT 'Programada',  
    procedimiento_realizado TEXT,  
    CONSTRAINT pk_cirugia PRIMARY KEY (cirugia_id),
```

```

CONSTRAINT uq_cirugia_solicitud UNIQUE (solicitud_cirugia_id),
CONSTRAINT uq_cirugia_evaluacion UNIQUE (evaluacion_solicitud_cirugia_id),
-- Solo la decision aprobada puede producir una cirugia.
CONSTRAINT ck_cirugia_aprobada CHECK (decision_requerida = 'Aprobada'),
-- Define los estados de la intervencion.
CONSTRAINT ck_cirugia_estado CHECK (
    estado IN ('Programada', 'En curso', 'Finalizada', 'Cancelada')
),
-- La finalizacion debe seguir al inicio de la intervencion.
CONSTRAINT ck_cirugia_fechas CHECK (
    (fecha_fin IS NULL OR fecha_inicio IS NOT NULL)
    AND (fecha_fin IS NULL OR fecha_fin >= fecha_inicio)
),
-- Una cirugia finalizada debe tener inicio, fin y procedimiento registrado.
CONSTRAINT ck_cirugia_finalizada CHECK (
    estado <> 'Finalizada' OR
    (fecha_inicio IS NOT NULL AND fecha_fin IS NOT NULL
    AND NULLIF(BTRIM(procedimiento_realizado), '') IS NOT NULL)
),
CONSTRAINT fk_cirugia_solicitud FOREIGN KEY (solicitud_cirugia_id)
    REFERENCES solicitud_cirugia (solicitud_cirugia_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_cirugia_evaluacion_aprobada
    FOREIGN KEY (evaluacion_solicitud_cirugia_id, solicitud_cirugia_id, decision_requerida)
    REFERENCES evaluacion_solicitud_cirugia
        (evaluacion_solicitud_cirugia_id, solicitud_cirugia_id, decision)
    ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_cirugia_quirofano FOREIGN KEY (quirofano_id)
    REFERENCES espacio_hospitalario (espacio_hospitalario_id)
    ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Resuelve la participacion de cirujanos, anestesistas y enfermeros.
CREATE TABLE participante_cirugia (
    participante_cirugia_id BIGINT GENERATED ALWAYS AS IDENTITY,
    cirugia_id BIGINT NOT NULL,
    persona_id BIGINT NOT NULL,
    rol VARCHAR(25) NOT NULL,
    observaciones TEXT,
    CONSTRAINT pk_participante_cirugia PRIMARY KEY (participante_cirugia_id),

```

```

CONSTRAINT uq_participante_cirugia_rol UNIQUE (cirugia_id, persona_id, rol),
-- Limita los roles del equipo quirurgico.
CONSTRAINT ck_participante_rol CHECK (rol IN (
    'Cirujano', 'Anestesiologo', 'Enfermero',
    'Practicante medicina', 'Practicante enfermeria', 'Otro'
)),
CONSTRAINT fk_participante_cirugia FOREIGN KEY (cirugia_id)
    REFERENCES cirugia (cirugia_id) ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_participante_persona FOREIGN KEY (persona_id)
    REFERENCES persona (persona_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Cataloga insumos, instrumentos y equipos utilizados en cirugia.
CREATE TABLE recurso (
    recurso_id BIGINT GENERATED ALWAYS AS IDENTITY,
    nombre VARCHAR(150) NOT NULL,
    descripcion TEXT,
    categoria VARCHAR(11) NOT NULL,
    tipo VARCHAR(12) NOT NULL,
    material VARCHAR(100),
    funcion VARCHAR(20),
    costo_unitario NUMERIC(12,2) NOT NULL DEFAULT 0,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_recurso PRIMARY KEY (recurso_id),
    CONSTRAINT uq_recurso_nombre_categoria UNIQUE (nombre, categoria),
    -- Diferencia insumo, instrumento y equipo.
    CONSTRAINT ck_recurso_categoria CHECK (
        categoria IN ('Insumo', 'Instrumento', 'Equipo')
    ),
    -- Limita el tipo de recurso descrito en las fichas.
    CONSTRAINT ck_recurso_tipo CHECK (tipo IN ('Medico', 'Quirurgico', 'Otro')),
    -- Limita las funciones posibles de instrumentos y equipos.
    CONSTRAINT ck_recurso_funcion CHECK (
        funcion IS NULL OR funcion IN (
            'Corte', 'Contenido', 'Hemostatica', 'Retractor',
            'Accesorio', 'Implante', 'Exploracion', 'Diagnostico',
            'Tratamiento', 'Rehabilitacion', 'Otro'
        )
    ),
    -- El costo unitario no puede ser negativo.

```

```

CONSTRAINT ck_recurso_costo CHECK (costo_unitario >= 0),
-- Exige un nombre de recurso.
CONSTRAINT ck_recurso_nombre CHECK (BTRIM(nombre) <> ''),
-- Exige material en insumos cuando corresponda a su ficha.
CONSTRAINT ck_recurso_material_insumo CHECK (
    categoria <> 'Insumo' OR NULLIF(BTRIM(material), '') IS NOT NULL
),
-- Exige funcion para instrumentos y equipos.
CONSTRAINT ck_recurso_funcion_equipo CHECK (
    categoria = 'Insumo' OR funcion IS NOT NULL
)
);

-- Relaciona la solicitud con los recursos requeridos en el agendamiento.
CREATE TABLE solicitud_recurso (
    solicitud_cirugia_id BIGINT NOT NULL,
    recurso_id BIGINT NOT NULL,
    cantidad NUMERIC(10,2) NOT NULL,
    observaciones TEXT,
    CONSTRAINT pk_solicitud_recurso PRIMARY KEY (solicitud_cirugia_id, recurso_id),
    -- Se deben solicitar cantidades positivas.
    CONSTRAINT ck_solicitud_recurso_cantidad CHECK (cantidad > 0),
    CONSTRAINT fk_solicitud_recurso_solicitud FOREIGN KEY (solicitud_cirugia_id)
        REFERENCES solicitud_cirugia (solicitud_cirugia_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_solicitud_recurso_recurso FOREIGN KEY (recurso_id)
        REFERENCES recurso (recurso_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Conserva cantidades y costos historicos de recursos usados realmente.
CREATE TABLE uso_recurso_cirugia (
    uso_recurso_cirugia_id BIGINT GENERATED ALWAYS AS IDENTITY,
    cirugia_id BIGINT NOT NULL,
    recurso_id BIGINT NOT NULL,
    cantidad NUMERIC(10,2) NOT NULL,
    costo_unitario_aplicado NUMERIC(12,2) NOT NULL,
    fecha_uso TIMESTAMPTZ NOT NULL,
    observaciones TEXT,
    CONSTRAINT pk_uso_recurso_cirugia PRIMARY KEY (uso_recurso_cirugia_id),
    -- El uso debe tener una cantidad positiva.

```

```

CONSTRAINT ck_uso_recurso_cantidad CHECK (cantidad > 0),
-- El costo aplicado no puede ser negativo.
CONSTRAINT ck_uso_recurso_costo CHECK (costo_unitario_aplicado >= 0),
CONSTRAINT fk_uso_recurso_cirugia FOREIGN KEY (cirugia_id)
    REFERENCES cirugia (cirugia_id) ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_uso_recurso_catalogo FOREIGN KEY (recurso_id)
    REFERENCES recurso (recurso_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Documenta el consentimiento informado especifico del procedimiento.
CREATE TABLE consentimiento_quirurgico (
    consentimiento_quirurgico_id BIGINT GENERATED ALWAYS AS IDENTITY,
    cirugia_id BIGINT NOT NULL,
    medico_firmante_id BIGINT NOT NULL,
    firmante_persona_id BIGINT NOT NULL,
    calidad_firmante VARCHAR(20) NOT NULL,
    procedimiento TEXT NOT NULL,
    objetivo TEXT NOT NULL,
    características TEXT NOT NULL,
    riesgos TEXT NOT NULL,
    fecha_obtencion TIMESTAMPTZ NOT NULL,
    firma_medico_confirmada BOOLEAN NOT NULL,
    firma_paciente_o_representante_confirmada BOOLEAN NOT NULL,
    CONSTRAINT pk_consentimiento_quirurgico PRIMARY KEY (consentimiento_quirurgico_id),
    CONSTRAINT uq_consentimiento_quirurgico_cirugia UNIQUE (cirugia_id),
    -- Identifica la calidad en que firma el consentimiento.
    CONSTRAINT ck_consentimiento_quirurgico_firmante CHECK (
        calidad_firmante IN ('Paciente', 'Familiar', 'Tutor', 'Representante')
    ),
    -- El procedimiento descrito debe contener texto.
    CONSTRAINT ck_consentimiento_quirurgico_procedimiento CHECK (BTRIM(procedimiento) <> ''),
    -- El objetivo del consentimiento debe contener texto.
    CONSTRAINT ck_consentimiento_quirurgico_objetivo CHECK (BTRIM(objetivo) <> ''),
    -- Las características del procedimiento deben contener texto.
    CONSTRAINT ck_consentimiento_quirurgico_características CHECK (BTRIM(características) <> ''),
    -- Los posibles riesgos deben documentarse.
    CONSTRAINT ck_consentimiento_quirurgico_riesgos CHECK (BTRIM(riesgos) <> ''),
    CONSTRAINT fk_consentimiento_quirurgico_cirugia FOREIGN KEY (cirugia_id)
        REFERENCES cirugia (cirugia_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_consentimiento_quirurgico_medico FOREIGN KEY (medico_firmante_id)

```

```

        REFERENCES medico (medico_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_consentimiento_quirurgico_persona FOREIGN KEY (firmante_persona_id)
        REFERENCES persona (persona_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

```

-- Registra la clasificacion ASA y el plan de anestesia preoperatorio.

```

CREATE TABLE chequeo_preanestesico (
    chequeo_preanestesico_id BIGINT GENERATED ALWAYS AS IDENTITY,
    cirugia_id BIGINT NOT NULL,
    medico_evaluador_id BIGINT NOT NULL,
    anestesista_id BIGINT NOT NULL,
    clasificacion_asa VARCHAR(3) NOT NULL,
    plan_anestesia TEXT NOT NULL,
    fecha_evaluacion TIMESTAMPTZ NOT NULL,
    firma_evaluador_confirmada BOOLEAN NOT NULL,
    observaciones TEXT,
    CONSTRAINT pk_chequeo_preanestesico PRIMARY KEY (chequeo_preanestesico_id),
    CONSTRAINT uq_chequeo_preanestesico_cirugia UNIQUE (cirugia_id),
    -- Restringe la clasificacion ASA a los grados I a VI.
    CONSTRAINT ck_chequeo_asa CHECK (
        clasificacion_asa IN ('I', 'II', 'III', 'IV', 'V', 'VI')
    ),
    -- Exige documentar el plan de anestesia.
    CONSTRAINT ck_chequeo_plan CHECK (BTRIM(plan_anestesia) <> ''),
    CONSTRAINT fk_chequeo_cirugia FOREIGN KEY (cirugia_id)
        REFERENCES cirugia (cirugia_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_chequeo_evaluador FOREIGN KEY (medico_evaluador_id)
        REFERENCES medico (medico_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_chequeo_anestesista FOREIGN KEY (anestesista_id)
        REFERENCES medico (medico_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

```

-- Registra preoperatorio, intraoperatorio y postoperatorio de cada cirugia.

```

CREATE TABLE etapa_cirugia (
    etapa_cirugia_id BIGINT GENERATED ALWAYS AS IDENTITY,
    cirugia_id BIGINT NOT NULL,
    enfermero_responsable_id BIGINT NOT NULL,
    etapa VARCHAR(16) NOT NULL,
    fecha_inicio TIMESTAMPTZ NOT NULL,
    fecha_fin TIMESTAMPTZ,

```

```

    fecha_envio_secretaria TIMESTAMPTZ,
    observaciones TEXT,
    CONSTRAINT pk_etapa_cirugia PRIMARY KEY (etapa_cirugia_id),
    CONSTRAINT uq_etapa_cirugia_tipo UNIQUE (cirugia_id, etapa),
    CONSTRAINT uq_etapa_cirugia_tipo_ref UNIQUE (etapa_cirugia_id, etapa),
    -- Limita el proceso a las tres etapas quirurgicas.
    CONSTRAINT ck_etapa_tipo CHECK (
        etapa IN ('Preoperatorio', 'Intraoperatorio', 'Postoperatorio')
    ),
    -- El cierre debe ser posterior al inicio.
    CONSTRAINT ck_etapa_fechas CHECK (fecha_fin IS NULL OR fecha_fin >= fecha_inicio),
    -- El envio a secretaria requiere etapa finalizada.
    CONSTRAINT ck_etapa_envio CHECK (
        fecha_envio_secretaria IS NULL OR
        (fecha_fin IS NOT NULL AND fecha_envio_secretaria >= fecha_fin)
    ),
    CONSTRAINT fk_etapa_cirugia FOREIGN KEY (cirugia_id)
        REFERENCES cirugia (cirugia_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_etapa_enfermero FOREIGN KEY (enfermero_responsable_id)
        REFERENCES enfermero (enfermero_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Cataloga verificaciones de cada fase del proceso quirurgico.
CREATE TABLE item_control_quirurgico (
    item_control_quirurgico_id BIGINT GENERATED ALWAYS AS IDENTITY,
    etapa VARCHAR(16) NOT NULL,
    grupo VARCHAR(40) NOT NULL,
    descripcion VARCHAR(250) NOT NULL,
    tipo_resultado VARCHAR(12) NOT NULL,
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    CONSTRAINT pk_item_control_quirurgico PRIMARY KEY (item_control_quirurgico_id),
    CONSTRAINT uq_item_control_grupo_descripcion UNIQUE (etapa, grupo, descripcion),
    CONSTRAINT uq_item_control_etapa_ref UNIQUE (item_control_quirurgico_id, etapa),
    -- Limita cada control a una etapa valida.
    CONSTRAINT ck_item_control_etapa CHECK (
        etapa IN ('Preoperatorio', 'Intraoperatorio', 'Postoperatorio')
    ),
    -- Distingue verificacion binaria de nivel de aceptacion.
    CONSTRAINT ck_item_control_tipo CHECK (tipo_resultado IN ('ExitoFalla', 'Aceptacion')),
    -- Exige identificar el grupo del control.

```

```

CONSTRAINT ck_item_control_grupo CHECK (BTRIM(grupo) <> ''),
-- Exige una descripcion del aspecto verificado.
CONSTRAINT ck_item_control_descripcion CHECK (BTRIM(descripcion) <> '')
);

-- Registra el resultado de cada verificacion y su hora.
CREATE TABLE resultado_control_quirurgico (
    resultado_control_quirurgico_id BIGINT GENERATED ALWAYS AS IDENTITY,
    etapa_cirugia_id BIGINT NOT NULL,
    item_control_quirurgico_id BIGINT NOT NULL,
    etapa VARCHAR(16) NOT NULL,
    resultado VARCHAR(25) NOT NULL,
    fecha_registro TIMESTAMPTZ NOT NULL,
    registrado_por_enfermero_id BIGINT NOT NULL,
    observaciones TEXT,
    CONSTRAINT pk_resultado_control PRIMARY KEY (resultado_control_quirurgico_id),
    CONSTRAINT uq_resultado_control_item UNIQUE (etapa_cirugia_id, item_control_quirurgico_id),
    -- Acepta las evaluaciones previstas por los protocolos quirurgicos.
    CONSTRAINT ck_resultado_control_valor CHECK (
        resultado IN ('Exito', 'Falla', 'Aceptable',
            'Medianamente aceptable', 'No aceptable')
    ),
    CONSTRAINT fk_resultado_etapa FOREIGN KEY (etapa_cirugia_id, etapa)
        REFERENCES etapa_cirugia (etapa_cirugia_id, etapa)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_resultado_item FOREIGN KEY (item_control_quirurgico_id, etapa)
        REFERENCES item_control_quirurgico (item_control_quirurgico_id, etapa)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_resultado_enfermero FOREIGN KEY (registrado_por_enfermero_id)
        REFERENCES enfermero (enfermero_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Conserva historia clinica, signos, exploraciones y cuidados de cirugia.
CREATE TABLE registro_clinico_cirugia (
    registro_clinico_cirugia_id BIGINT GENERATED ALWAYS AS IDENTITY,
    cirugia_id BIGINT NOT NULL,
    etapa_cirugia_id BIGINT,
    registrador_persona_id BIGINT NOT NULL,
    categoria VARCHAR(32) NOT NULL,
    descripcion TEXT NOT NULL,

```

```

valor TEXT,

fecha_registro TIMESTAMPTZ NOT NULL,

CONSTRAINT pk_registro_clinico_cirugia PRIMARY KEY (registro_clinico_cirugia_id),

-- Clasifica la informacion clinica obtenida en las etapas.

CONSTRAINT ck_registro_clinico_categoria CHECK (categoria IN (

    'Interrogatorio', 'Antecedentes', 'Exploracion fisica',

    'Signos vitales', 'Balance hidrico', 'Medicamentos',

    'Transfusion', 'Evolucion', 'Otro'

)),

-- El registro debe describir el hallazgo o cuidado.

CONSTRAINT ck_registro_clinico_descripcion CHECK (BTRIM(descripcion) <> ''),

CONSTRAINT fk_registro_clinico_cirugia FOREIGN KEY (cirugia_id)

    REFERENCES cirugia (cirugia_id) ON DELETE RESTRICT ON UPDATE CASCADE,

CONSTRAINT fk_registro_clinico_etapa FOREIGN KEY (etapa_cirugia_id)

    REFERENCES etapa_cirugia (etapa_cirugia_id)

    ON DELETE RESTRICT ON UPDATE CASCADE,

CONSTRAINT fk_registro_clinico_persona FOREIGN KEY (registrador_persona_id)

    REFERENCES persona (persona_id) ON DELETE RESTRICT ON UPDATE CASCADE

);

COMMIT;

/*

Verificacion: 40 tablas anteriores + 13 tablas de cirugia = 53.

SELECT COUNT(*) FROM information_schema.tables

WHERE table_schema = 'hospital' AND table_type = 'BASE TABLE';

*/

```

hospitalizacion.sql

```
BEGIN;

SET search_path TO hospital, public;

-- Registra una estadia hospitalaria vinculada a la ficha de ingreso.

CREATE TABLE hospitalizacion (
    hospitalizacion_id BIGINT GENERATED ALWAYS AS IDENTITY,
    ingreso_id BIGINT NOT NULL,
    medico_encargado_id BIGINT NOT NULL,
    fecha_alta_prevista TIMESTAMPTZ,
    fecha_alta_real TIMESTAMPTZ,
    costo_diario_aplicado NUMERIC(12,2) NOT NULL,
    observaciones TEXT,
    CONSTRAINT pk_hospitalizacion PRIMARY KEY (hospitalizacion_id),
    CONSTRAINT uq_hospitalizacion_ingreso UNIQUE (ingreso_id),
    -- El costo diario historico no puede ser negativo.
    CONSTRAINT ck_hospitalizacion_costo CHECK (costo_diario_aplicado >= 0),
    CONSTRAINT fk_hospitalizacion_ingreso FOREIGN KEY (ingreso_id)
        REFERENCES ingreso (ingreso_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_hospitalizacion_medico FOREIGN KEY (medico_encargado_id)
        REFERENCES medico (medico_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Conserva las camas utilizadas y sus intervalos en una hospitalizacion.

CREATE TABLE asignacion_cama (
    asignacion_cama_id BIGINT GENERATED ALWAYS AS IDENTITY,
    hospitalizacion_id BIGINT NOT NULL,
    cama_id BIGINT NOT NULL,
    fecha_inicio TIMESTAMPTZ NOT NULL,
    fecha_fin TIMESTAMPTZ,
    observaciones TEXT,
    CONSTRAINT pk_asignacion_cama PRIMARY KEY (asignacion_cama_id),
    -- El fin de ocupacion debe ser posterior al inicio.
    CONSTRAINT ck_asignacion_cama_fechas CHECK (
        fecha_fin IS NULL OR fecha_fin > fecha_inicio
    ),
    CONSTRAINT fk_asignacion_cama_hospitalizacion FOREIGN KEY (hospitalizacion_id)
        REFERENCES hospitalizacion (hospitalizacion_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_asignacion_cama_espacio FOREIGN KEY (cama_id)
```

```

        REFERENCES espacio_hospitalario (espacio_hospitalario_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
    );

-- Registra atenciones y procedimientos prestados durante la estadia.
CREATE TABLE hospitalizacion_servicio (
    hospitalizacion_servicio_id BIGINT GENERATED ALWAYS AS IDENTITY,
    hospitalizacion_id BIGINT NOT NULL,
    servicio_medico_id SMALLINT NOT NULL,
    medico_id BIGINT NOT NULL,
    fecha_servicio TIMESTAMPTZ NOT NULL,
    descripcion TEXT NOT NULL,
    costo_aplicado NUMERIC(12,2) NOT NULL,
    CONSTRAINT pk_hospitalizacion_servicio PRIMARY KEY (hospitalizacion_servicio_id),
    -- El servicio requiere una descripcion.
    CONSTRAINT ck_hospitalizacion_servicio_descripcion CHECK (BTRIM(descripcion) <> ''),
    -- El costo del servicio no puede ser negativo.
    CONSTRAINT ck_hospitalizacion_servicio_costo CHECK (costo_aplicado >= 0),
    CONSTRAINT fk_hospitalizacion_servicio_estadia FOREIGN KEY (hospitalizacion_id)
        REFERENCES hospitalizacion (hospitalizacion_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_hospitalizacion_servicio_catalogo FOREIGN KEY (servicio_medico_id)
        REFERENCES servicio_medico (servicio_medico_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_hospitalizacion_servicio_medico FOREIGN KEY (medico_id)
        REFERENCES medico (medico_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Conserva cantidades y costos de insumos usados en hospitalizacion.
CREATE TABLE hospitalizacion_insumo (
    hospitalizacion_insumo_id BIGINT GENERATED ALWAYS AS IDENTITY,
    hospitalizacion_id BIGINT NOT NULL,
    recurso_id BIGINT NOT NULL,
    fecha_uso TIMESTAMPTZ NOT NULL,
    cantidad NUMERIC(10,2) NOT NULL,
    costo_unitario_aplicado NUMERIC(12,2) NOT NULL,
    observaciones TEXT,
    CONSTRAINT pk_hospitalizacion_insumo PRIMARY KEY (hospitalizacion_insumo_id),
    -- Se exige una cantidad positiva de insumos.
    CONSTRAINT ck_hospitalizacion_insumo_cantidad CHECK (cantidad > 0),

```

```
-- El costo unitario historico no puede ser negativo.
CONSTRAINT ck_hospitalizacion_insumo_costo CHECK (costo_unitario_aplicado >= 0),
CONSTRAINT fk_hospitalizacion_insumo_estadia FOREIGN KEY (hospitalizacion_id)
    REFERENCES hospitalizacion (hospitalizacion_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_hospitalizacion_insumo_recurso FOREIGN KEY (recurso_id)
    REFERENCES recurso (recurso_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Un mismo paciente solo puede tener una asignacion de cama activa por estadia.
CREATE UNIQUE INDEX uq_asignacion_activa_hospitalizacion
    ON asignacion_cama (hospitalizacion_id) WHERE fecha_fin IS NULL;

-- Una cama solo puede tener una asignacion activa.
CREATE UNIQUE INDEX uq_asignacion_activa_cama
    ON asignacion_cama (cama_id) WHERE fecha_fin IS NULL;

COMMIT;
```

facturacion_calidad.sql

```
/*  
  
Proyecto 1 - Bloque 6: facturacion, cuotas, pagos y calidad.  
  
Ejecutar despues de hospitalizacion.sql.  
  
*/  
  
BEGIN;  
  
SET search_path TO hospital, public;  
  
  
-- Registra el comprobante de una consulta o de un ingreso hospitalario.  
CREATE TABLE factura (  
  
    factura_id BIGINT GENERATED ALWAYS AS IDENTITY,  
    numero_factura VARCHAR(30) NOT NULL,  
    paciente_id BIGINT NOT NULL,  
    hospital_id BIGINT NOT NULL,  
    ingreso_id BIGINT,  
    consulta_externa_id BIGINT,  
    fecha_emision TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    descripcion VARCHAR(300) NOT NULL,  
    estado VARCHAR(12) NOT NULL DEFAULT 'Pendiente',  
    CONSTRAINT pk_factura PRIMARY KEY (factura_id),  
    CONSTRAINT uq_factura_numero UNIQUE (numero_factura),  
    CONSTRAINT uq_factura_ingreso UNIQUE (ingreso_id),  
    CONSTRAINT uq_factura_consulta UNIQUE (consulta_externa_id),  
    -- Una factura corresponde a una consulta o un ingreso, exclusivamente.  
    CONSTRAINT ck_factura_origen CHECK (  
        (ingreso_id IS NOT NULL AND consulta_externa_id IS NULL)  
        OR (ingreso_id IS NULL AND consulta_externa_id IS NOT NULL)  
    ),  
    -- Limita los estados del comprobante.  
    CONSTRAINT ck_factura_estado CHECK (estado IN ('Pendiente', 'Parcial', 'Pagada', 'Anulada')),  
    -- Exige una descripcion breve y clara del servicio.  
    CONSTRAINT ck_factura_descripcion CHECK (BTRIM(descripcion) <> ''),  
    CONSTRAINT fk_factura_paciente FOREIGN KEY (paciente_id)  
        REFERENCES paciente (paciente_id) ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_factura_hospital FOREIGN KEY (hospital_id)  
        REFERENCES hospital (hospital_id) ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_factura_ingreso FOREIGN KEY (ingreso_id)  
        REFERENCES ingreso (ingreso_id) ON DELETE RESTRICT ON UPDATE CASCADE,  
    CONSTRAINT fk_factura_consulta FOREIGN KEY (consulta_externa_id)
```

```

        REFERENCES consulta_externa (consulta_externa_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
    );

-- Desglosa los servicios e insumos con precio historico aplicado.
CREATE TABLE factura_detalle (
    factura_detalle_id BIGINT GENERATED ALWAYS AS IDENTITY,
    factura_id BIGINT NOT NULL,
    concepto VARCHAR(250) NOT NULL,
    cantidad NUMERIC(10,2) NOT NULL,
    precio_unitario NUMERIC(12,2) NOT NULL,
    porcentaje_descuento NUMERIC(5,2) NOT NULL DEFAULT 0,
    porcentaje_recargo NUMERIC(5,2) NOT NULL DEFAULT 0,
    subtotal NUMERIC(14,2) GENERATED ALWAYS AS (
        ROUND(cantidad * precio_unitario *
            (1 - porcentaje_descuento / 100 + porcentaje_recargo / 100), 2)
    ) STORED,
    CONSTRAINT pk_factura_detalle PRIMARY KEY (factura_detalle_id),
    -- Cada detalle necesita un concepto legible.
    CONSTRAINT ck_factura_detalle_concepto CHECK (BTRIM(concepto) <> ''),
    -- La cantidad facturada debe ser positiva.
    CONSTRAINT ck_factura_detalle_cantidad CHECK (cantidad > 0),
    -- El precio no puede ser negativo.
    CONSTRAINT ck_factura_detallePrecio CHECK (precio_unitario >= 0),
    -- El descuento debe estar entre cero y cien por ciento.
    CONSTRAINT ck_factura_detalle_descuento CHECK (porcentaje_descuento BETWEEN 0 AND 100),
    -- El recargo debe estar entre cero y cien por ciento.
    CONSTRAINT ck_factura_detalle_recargo CHECK (porcentaje_recargo BETWEEN 0 AND 100),
    CONSTRAINT fk_factura_detalle_factura FOREIGN KEY (factura_id)
        REFERENCES factura (factura_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- Permite distribuir el costo de un ingreso en una a doce cuotas.
CREATE TABLE plan_pago (
    plan_pago_id BIGINT GENERATED ALWAYS AS IDENTITY,
    factura_id BIGINT NOT NULL,
    numero_cuotas SMALLINT NOT NULL,
    fecha_inicio DATE NOT NULL,
    estado VARCHAR(12) NOT NULL DEFAULT 'Activo',
    CONSTRAINT pk_plan_pago PRIMARY KEY (plan_pago_id),

```

```

CONSTRAINT uq_plan_pago_factura UNIQUE (factura_id),
CONSTRAINT uq_plan_pago_factura_ref UNIQUE (plan_pago_id, factura_id),
-- Limita la financiacion a un maximo de doce pagos mensuales.
CONSTRAINT ck_plan_pago_cuotas CHECK (numero_cuotas BETWEEN 1 AND 12),
-- Controla el ciclo de vida del plan.
CONSTRAINT ck_plan_pago_estado CHECK (estado IN ('Activo', 'Liquidado', 'Cancelado')),
CONSTRAINT fk_plan_pago_factura FOREIGN KEY (factura_id)
    REFERENCES factura (factura_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

```

-- Registra los vencimientos y montos de un plan de pago.

```

CREATE TABLE cuota (
    cuota_id BIGINT GENERATED ALWAYS AS IDENTITY,
    plan_pago_id BIGINT NOT NULL,
    numero SMALLINT NOT NULL,
    fecha_vencimiento DATE NOT NULL,
    monto NUMERIC(12,2) NOT NULL,
    estado VARCHAR(12) NOT NULL DEFAULT 'Pendiente',
    CONSTRAINT pk_cuota PRIMARY KEY (cuota_id),
    CONSTRAINT uq_cuota_numero UNIQUE (plan_pago_id, numero),
    CONSTRAINT uq_cuota_plan_ref UNIQUE (cuota_id, plan_pago_id),
    -- La numeracion de cuotas debe estar en el intervalo permitido.
    CONSTRAINT ck_cuota_numero CHECK (numero BETWEEN 1 AND 12),
    -- Cada cuota requiere un monto mayor que cero.
    CONSTRAINT ck_cuota_monto CHECK (monto > 0),
    -- Controla el estado individual de la cuota.
    CONSTRAINT ck_cuota_estado CHECK (estado IN ('Pendiente', 'Parcial', 'Pagada')),
    CONSTRAINT fk_cuota_plan FOREIGN KEY (plan_pago_id)
        REFERENCES plan_pago (plan_pago_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

```

-- Documenta los pagos recibidos y la recepcion donde se cobraron.

```

CREATE TABLE pago (
    pago_id BIGINT GENERATED ALWAYS AS IDENTITY,
    factura_id BIGINT NOT NULL,
    plan_pago_id BIGINT,
    cuota_id BIGINT,
    unidad_recepcion_id BIGINT NOT NULL,
    fecha_pago TIMESTAMPTZ NOT NULL,
    monto NUMERIC(12,2) NOT NULL,

```

```

metodo VARCHAR(15) NOT NULL,
referencia VARCHAR(80),
CONSTRAINT pk_pago PRIMARY KEY (pago_id),
-- Los pagos deben tener un monto positivo.
CONSTRAINT ck_pago_monto CHECK (monto > 0),
-- Limita los medios de pago aceptados.
CONSTRAINT ck_pago_metodo CHECK (
    metodo IN ('Efectivo', 'Tarjeta', 'Transferencia', 'Deposito')
),
-- Una cuota solo puede asociarse a un plan informado.
CONSTRAINT ck_pago_plan_cuota CHECK (cuota_id IS NULL OR plan_pago_id IS NOT NULL),
CONSTRAINT fk_pago_factura FOREIGN KEY (factura_id)
    REFERENCES factura (factura_id) ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_pago_plan_factura FOREIGN KEY (plan_pago_id, factura_id)
    REFERENCES plan_pago (plan_pago_id, factura_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_pago_cuota_plan FOREIGN KEY (cuota_id, plan_pago_id)
    REFERENCES cuota (cuota_id, plan_pago_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_pago_recepcion FOREIGN KEY (unidad_recepcion_id)
    REFERENCES unidad_medica (unidad_medica_id)
    ON DELETE RESTRICT ON UPDATE CASCADE
);

```

-- Recoge la opinion de un paciente sobre su atencion en un hospital.

```

CREATE TABLE evaluacion_calidad (
    evaluacion_calidad_id BIGINT GENERATED ALWAYS AS IDENTITY,
    paciente_id BIGINT NOT NULL,
    hospital_id BIGINT NOT NULL,
    ingreso_id BIGINT,
    consulta_externa_id BIGINT,
    fecha_evaluacion TIMESTAMPTZ NOT NULL,
    observaciones TEXT,
    CONSTRAINT pk_evaluacion_calidad PRIMARY KEY (evaluacion_calidad_id),
-- La evaluacion corresponde a un ingreso o a una consulta.
CONSTRAINT ck_evaluacion_calidad_origen CHECK (
    (ingreso_id IS NOT NULL AND consulta_externa_id IS NULL)
    OR (ingreso_id IS NULL AND consulta_externa_id IS NOT NULL)
),
CONSTRAINT fk_calidad_paciente FOREIGN KEY (paciente_id)

```

```

        REFERENCES paciente (paciente_id) ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_calidad_hospital FOREIGN KEY (hospital_id)
        REFERENCES hospital (hospital_id) ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_calidad_ingreso FOREIGN KEY (ingreso_id)
        REFERENCES ingreso (ingreso_id) ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_calidad_consulta FOREIGN KEY (consulta_externa_id)
        REFERENCES consulta_externa (consulta_externa_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
);

```

-- Califica a un hospital o a una persona participante de la atencion.

```

CREATE TABLE evaluacion_objetivo (
    evaluacion_objetivo_id BIGINT GENERATED ALWAYS AS IDENTITY,
    evaluacion_calidad_id BIGINT NOT NULL,
    hospital_objetivo_id BIGINT,
    persona_objetivo_id BIGINT,
    nota SMALLINT NOT NULL,
    comentario TEXT,
    CONSTRAINT pk_evaluacion_objetivo PRIMARY KEY (evaluacion_objetivo_id),
    -- Solo se puede calificar uno de los dos tipos de objetivo por fila.
    CONSTRAINT ck_evaluacion_objetivo_unico CHECK (
        (hospital_objetivo_id IS NOT NULL AND persona_objetivo_id IS NULL)
        OR (hospital_objetivo_id IS NULL AND persona_objetivo_id IS NOT NULL)
    ),
    -- La calificacion va de uno a cinco.
    CONSTRAINT ck_evaluacion_nota CHECK (nota BETWEEN 1 AND 5),
    CONSTRAINT fk_objetivo_evaluacion FOREIGN KEY (evaluacion_calidad_id)
        REFERENCES evaluacion_calidad (evaluacion_calidad_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_objetivo_hospital FOREIGN KEY (hospital_objetivo_id)
        REFERENCES hospital (hospital_id) ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_objetivo_persona FOREIGN KEY (persona_objetivo_id)
        REFERENCES persona (persona_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

```

-- Evita calificar dos veces al mismo hospital en una evaluacion.

```

CREATE UNIQUE INDEX uq_calidad_hospital_objetivo
    ON evaluacion_objetivo (evaluacion_calidad_id, hospital_objetivo_id)
    WHERE hospital_objetivo_id IS NOT NULL;

```

```
-- Evita calificar dos veces a la misma persona en una evaluacion.  
  
CREATE UNIQUE INDEX uq_calidad_persona_objetivo  
    ON evaluacion_objetivo (evaluacion_calidad_id, persona_objetivo_id)  
    WHERE persona_objetivo_id IS NOT NULL;  
  
COMMIT;
```

datos_prueba.sql

```
BEGIN;

SET search_path TO hospital, public;

DO $$

DECLARE

    v_dep SMALLINT; v_mun INTEGER; v_dir BIGINT; v_hosp BIGINT;

    v_ce BIGINT; v_urg BIGINT; v_cir BIGINT; v_hospit BIGINT;

    v_sce SMALLINT; v_surg SMALLINT; v_scir SMALLINT; v_shosp SMALLINT;

    v_clinica BIGINT; v_camilla BIGINT; v_quirofano BIGINT; v_cama BIGINT;

    v_p1 BIGINT; v_p2 BIGINT; v_p3 BIGINT; v_p4 BIGINT;

    v_m1 BIGINT; v_m2 BIGINT; v_enfermero_persona BIGINT; v_empleado BIGINT;

    v_cita BIGINT; v_consulta BIGINT; v_receta BIGINT; v_orden BIGINT;

    v_ep2 BIGINT; v_ep3 BIGINT; v_ep4 BIGINT;

    v_i2 BIGINT; v_i3 BIGINT; v_i4 BIGINT; v_emergencia BIGINT;

    v_solicitud BIGINT; v_evaluacion BIGINT; v_cirugia BIGINT;

    v_estadia BIGINT; v_recurso BIGINT; v_diag BIGINT; v_egreso BIGINT;

    v_f BIGINT; v_plan BIGINT; v_cuota BIGINT; v_calidad BIGINT;

    v_especialidad SMALLINT; v_medicamento BIGINT;

    v_proc BIGINT; v_item BIGINT; v_etapa BIGINT;

BEGIN

    -- Ubicacion y establecimiento ficticio.

    INSERT INTO departamento (nombre) VALUES ('Departamento de Ensayo')

        RETURNING departamento_id INTO v_dep;

    INSERT INTO municipio (departamento_id, nombre)

        VALUES (v_dep, 'Municipio Modelo') RETURNING municipio_id INTO v_mun;

    INSERT INTO direccion (municipio_id, detalle, area)

        VALUES (v_mun, 'Avenida Ficticia 10, sector de pruebas', 'Urbana')

        RETURNING direccion_id INTO v_dir;

    INSERT INTO hospital (direccion_id, codigo, nombre, telefono)

        VALUES (v_dir, 'HOSP-DEMO-01', 'Hospital Modelo de Occidente', '55550000')

        RETURNING hospital_id INTO v_hosp;

    INSERT INTO unidad_medica (hospital_id, nombre, tipo)

        VALUES (v_hosp, 'Consulta Externa Modelo', 'Consulta Externa')

        RETURNING unidad_medica_id INTO v_ce;

    INSERT INTO unidad_medica (hospital_id, nombre, tipo)

        VALUES (v_hosp, 'Emergencias Modelo', 'Emergencias')

        RETURNING unidad_medica_id INTO v_urg;
```

```

INSERT INTO unidad_medica (hospital_id, nombre, tipo)
VALUES (v_hosp, 'Cirugia Modelo', 'Cirugia')
RETURNING unidad_medica_id INTO v_cir;

INSERT INTO unidad_medica (hospital_id, nombre, tipo)
VALUES (v_hosp, 'Hospitalizacion Modelo', 'Hospitalizacion')
RETURNING unidad_medica_id INTO v_hospit;

INSERT INTO servicio_medico (nombre, categoria)
VALUES ('Consulta general de demostracion', 'Consulta Externa')
RETURNING servicio_medico_id INTO v_sce;

INSERT INTO servicio_medico (nombre, categoria)
VALUES ('Estabilizacion de demostracion', 'Emergencias')
RETURNING servicio_medico_id INTO v_surg;

INSERT INTO servicio_medico (nombre, categoria)
VALUES ('Procedimiento quirurgico de demostracion', 'Cirugia')
RETURNING servicio_medico_id INTO v_scir;

INSERT INTO servicio_medico (nombre, categoria)
VALUES ('Estadia de demostracion', 'Hospitalizacion')
RETURNING servicio_medico_id INTO v_shosp;

INSERT INTO unidad_servicio (unidad_medica_id, servicio_medico_id, costo_base)
VALUES (v_ce,v_sce,200), (v_urg,v_surg,500),
(v_cir,v_scir,2500), (v_hospit,v_shosp,350);

INSERT INTO espacio_hospitalario (hospital_id, unidad_medica_id, codigo, tipo)
VALUES (v_hosp,v_ce,'CL-01','Clinica')
RETURNING espacio_hospitalario_id INTO v_clinica;

INSERT INTO espacio_hospitalario (hospital_id, unidad_medica_id, codigo, tipo)
VALUES (v_hosp,v_urg,'CAM-01','Camilla')
RETURNING espacio_hospitalario_id INTO v_camilla;

INSERT INTO espacio_hospitalario (hospital_id, unidad_medica_id, codigo, tipo)
VALUES (v_hosp,v_cir,'QX-01','Quirofano')
RETURNING espacio_hospitalario_id INTO v_quirofano;

INSERT INTO espacio_hospitalario
(hospital_id, unidad_medica_id, codigo, tipo, costo_diario)
VALUES (v_hosp,v_hospit,'CAMA-01','Cama',350)
RETURNING espacio_hospitalario_id INTO v_cama;

-- Cuatro pacientes ficticios, dos medicos y una enfermera.

INSERT INTO persona (direccion_id,nombres,apellidos,dpi,fecha_nacimiento,sexo,telefono)
VALUES (v_dir,'Alma','Ejemplo Uno','9000000000001','1998-04-10','Femenino','55550001')

```

```

RETURNING persona_id INTO v_p1;

INSERT INTO persona (direccion_id,nombres,apellidos,dpi,fecha_nacimiento,sexo,telefono)
VALUES (v_dir,'Bruno','Ejemplo Dos','9000000000002','1987-06-21','Masculino','55550002')
RETURNING persona_id INTO v_p2;

INSERT INTO persona (direccion_id,nombres,apellidos,dpi,fecha_nacimiento,sexo,telefono)
VALUES (v_dir,'Clara','Ejemplo Tres','9000000000003','2001-11-03','Femenino','55550003')
RETURNING persona_id INTO v_p3;

INSERT INTO persona (direccion_id,nombres,apellidos,dpi,fecha_nacimiento,sexo,telefono)
VALUES (v_dir,'Dario','Ejemplo Cuatro','9000000000004','1979-02-14','Masculino','55550004')
RETURNING persona_id INTO v_p4;

INSERT INTO paciente (paciente_id,numero_expediente)
VALUES (v_p1,'EXP-001'),(v_p2,'EXP-002'),(v_p3,'EXP-003'),(v_p4,'EXP-004');

INSERT INTO persona (direccion_id,nombres,apellidos,dpi,fecha_nacimiento,sexo)
VALUES (v_dir,'Elena','Medica Ficticia','9000000000005','1980-03-03','Femenino')
RETURNING persona_id INTO v_m1;

INSERT INTO persona (direccion_id,nombres,apellidos,dpi,fecha_nacimiento,sexo)
VALUES (v_dir,'Fabian','Cirujano Ficticio','9000000000006','1975-08-19','Masculino')
RETURNING persona_id INTO v_m2;

INSERT INTO medico (medico_id,numero_colegiado,clase_medico)
VALUES (v_m1,'COL-DEMO-01','Interno'),(v_m2,'COL-DEMO-02','Residente');

INSERT INTO medico_hospital (medico_id,hospital_id,condicion,fecha_inicio)
VALUES (v_m1,v_hosp,'Interno','2026-01-01'),
(v_m2,v_hosp,'Residente','2026-01-01');

INSERT INTO especialidad (nombre) VALUES ('Medicina General de Prueba')
RETURNING especialidad_id INTO v_especialidad;

INSERT INTO medico_especialidad (medico_id,especialidad_id,es_principal)
VALUES (v_m1,v_especialidad,TRUE);

INSERT INTO especialidad (nombre) VALUES ('Cirugia General de Prueba')
RETURNING especialidad_id INTO v_especialidad;

INSERT INTO medico_especialidad (medico_id,especialidad_id,es_principal)
VALUES (v_m2,v_especialidad,TRUE);

INSERT INTO persona (direccion_id,nombres,apellidos,dpi,fecha_nacimiento,sexo)
VALUES (v_dir,'Gabriela','Enfermera Ficticia','9000000000007','1990-09-09','Femenino')
RETURNING persona_id INTO v_enfermero_persona;

INSERT INTO empleado (persona_id,hospital_id,codigo_empleado,tipo_empleado,fecha_contratacion)
VALUES (v_enfermero_persona,v_hosp,'ENF-DEMO-01','Enfermero','2026-01-01')
RETURNING empleado_id INTO v_empleado;

INSERT INTO enfermero (enfermero_id,numero_registro,nivel)
VALUES (v_empleado,'REG-DEMO-01','Registrado');

```

```

-- Consulta externa: cita realizada, diagnostico, receta, laboratorio y pago.
INSERT INTO horario_consulta (medico_id,unidad_medica_id,clinica_id,dia_semana,hora_inicio,hora_fin)
VALUES (v_m1,v_ce,v_clinica,2,'08:00','12:00');

INSERT INTO cita
(paciente_id,medico_id,unidad_medica_id,clinica_id,fecha_hora,canal,tipo,estado,precio_base)
VALUES (v_p1,v_m1,v_ce,v_clinica,'2026-09-15 09:00:00-06','Telefono',
'Primera','Realizada',200)

RETURNING cita_id INTO v_cita;

INSERT INTO historial_estado_cita (cita_id,estado_nuevo)
VALUES (v_cita,'Programada');

INSERT INTO historial_estado_cita (cita_id,estado_anterior,estado_nuevo)
VALUES (v_cita,'Programada','Realizada');

INSERT INTO consulta_externa (cita_id,fecha_atencion,diagnostico,orientacion_paciente)
VALUES (v_cita,'2026-09-15 09:10:00-06',
'Evaluacion general ficticia sin hallazgos criticos',
'Orientacion preventiva de demostracion')

RETURNING consulta_externa_id INTO v_consulta;

INSERT INTO medicamento (nombre,presentacion)
VALUES ('Medicamento Modelo','Tableta ficticia')

RETURNING medicamento_id INTO v_medicamento;

INSERT INTO receta (consulta_externa_id,fecha_emision,fecha_proxima_cita)
VALUES (v_consulta,'2026-09-15','2026-10-15')

RETURNING receta_id INTO v_receta;

INSERT INTO receta_detalle (receta_id,medicamento_id,dosis,frecuencia,duracion_dias)
VALUES (v_receta,v_medicamento,'Una tableta','Cada 24 horas',3);

INSERT INTO orden_laboratorio (consulta_externa_id,fecha_emision)
VALUES (v_consulta,'2026-09-15 09:20:00-06')

RETURNING orden_laboratorio_id INTO v_orden;

INSERT INTO orden_laboratorio_detalle (orden_laboratorio_id,examen)
VALUES (v_orden,'Examen general ficticio');

INSERT INTO factura (numero_factura,paciente_id,hospital_id,consulta_externa_id,descripcion)
VALUES ('FAC-DEMO-001',v_p1,v_hosp,v_consulta,'Consulta externa de demostracion')

RETURNING factura_id INTO v_f;

INSERT INTO factura_detalle (factura_id,concepto,cantidad,precio_unitario)
VALUES (v_f,'Primera consulta',1,200);

INSERT INTO pago (factura_id,unidad_recepcion_id,fecha_pago,monto,metodo)
VALUES (v_f,v_ce,'2026-09-15 09:45:00-06',200,'Efectivo');

UPDATE factura SET estado='Pagada' WHERE factura_id=v_f;

```

```

-- Emergencia: ingreso, triaje, procedimiento, egreso y dos cuotas.
INSERT INTO episodio_atencion (paciente_id,hospital_id,fecha_apertura,motivo_apertura)
VALUES (v_p2,v_hosp,'2026-09-16 07:00:00-06','Atencion urgente simulada')
RETURNING episodio_atencion_id INTO v_ep2;
INSERT INTO ingreso (episodio_atencion_id,hospital_id,unidad_medica_id,
servicio_medico_id,medico_encargado_id,espacio_hospitalario_id,
fecha_ingreso,motivo_ingreso,diagnostico_presuntivo)
VALUES (v_ep2,v_hosp,v_urg,v_surg,v_m1,v_camilla,
'2026-09-16 07:05:00-06','Evaluacion inmediata ficticia',
'Necesidad de observacion clinica')
RETURNING ingreso_id INTO v_i2;
INSERT INTO atencion_emergencia (ingreso_id,fecha_evaluacion,prioridad,estado_clinico,resultado)
VALUES (v_i2,'2026-09-16 07:07:00-06',2,
'Paciente ficticio consciente','Estabilizado')
RETURNING atencion_emergencia_id INTO v_emergencia;
INSERT INTO procedimiento_emergencia (nombre,tipo,costo_base)
VALUES ('Observacion y estabilizacion ficticia','Basico',500)
RETURNING procedimiento_emergencia_id INTO v_proc;
INSERT INTO emergencia_procedimiento
(atencion_emergencia_id,procedimiento_emergencia_id,medico_id,
fecha_realizacion,costo_aplicado)
VALUES (v_emergencia,v_proc,v_m1,'2026-09-16 07:30:00-06',500);
INSERT INTO diagnostico (codigo,nombre)
VALUES ('DEMO-001','Diagnostico clinico ficticio')
RETURNING diagnostico_id INTO v_diag;
INSERT INTO egreso (ingreso_id,medico_asignado_id,fecha_egreso,motivo_egreso,codigo_egreso)
VALUES (v_i2,v_m1,'2026-09-16 11:00:00-06','Alta tras estabilizacion','Vivo')
RETURNING egreso_id INTO v_egreso;
INSERT INTO egreso_diagnostico (egreso_id,diagnostico_id,tipo)
VALUES (v_egreso,v_diag,'Principal');
UPDATE ingreso SET estado='Cerrado' WHERE ingreso_id=v_i2;
UPDATE episodio_atencion
SET estado='Cerrado',fecha_cierre='2026-09-16 11:00:00-06'
WHERE episodio_atencion_id=v_ep2;
INSERT INTO factura (numero_factura,paciente_id,hospital_id,ingreso_id,descripcion)
VALUES ('FAC-DEMO-002',v_p2,v_hosp,v_i2,'Servicio de emergencias de demostracion')
RETURNING factura_id INTO v_f;
INSERT INTO factura_detalle (factura_id,concepto,cantidad,precio_unitario)
VALUES (v_f,'Estabilizacion',1,500);
INSERT INTO plan_pago (factura_id,numero_cuotas,fecha_inicio)

```

```

VALUES (v_f,2,'2026-09-16') RETURNING plan_pago_id INTO v_plan;
INSERT INTO cuota (plan_pago_id,numero,fecha_vencimiento,monto)
VALUES (v_plan,1,'2026-09-16',250) RETURNING cuota_id INTO v_cuota;
INSERT INTO pago (factura_id,plan_pago_id,cuota_id,unidad_recepcion_id,
fecha_pago,monto,metodo)
VALUES (v_f,v_plan,v_cuota,v_hospit,'2026-09-16 11:10:00-06',250,'Efectivo');
UPDATE cuota SET estado='Pagada' WHERE cuota_id=v_cuota;
UPDATE factura SET estado='Parcial' WHERE factura_id=v_f;
INSERT INTO cuota (plan_pago_id,numero,fecha_vencimiento,monto)
VALUES (v_plan,2,'2026-10-16',250);

-- Cirugia: solicitud aprobada, quirofano, equipo y control preoperatorio.
INSERT INTO episodio_atencion (paciente_id,hospital_id,fecha_apertura,motivo_apertura)
VALUES (v_p3,v_hosp,'2026-09-17 07:00:00-06','Procedimiento simulado')
RETURNING episodio_atencion_id INTO v_ep3;
INSERT INTO solicitud_cirugia
(paciente_id,hospital_id,cirujano_id,episodio_atencion_id,fecha_solicitud,
caracter,historia_clinica,procedimiento_propuesto,duracion_estimada_minutos,tipo_anestesia)
VALUES (v_p3,v_hosp,v_m2,v_ep3,'2026-09-17 07:10:00-06','Programado',
'Antecedentes ficticios revisados','Intervencion simulada',60,'General')
RETURNING solicitud_cirugia_id INTO v_solicitud;
INSERT INTO evaluacion_solicitud_cirugia
(solicitud_cirugia_id,medico_evaluador_id,fecha_evaluacion,decision)
VALUES (v_solicitud,v_m1,'2026-09-17 07:30:00-06','Aprobada')
RETURNING evaluacion_solicitud_cirugia_id INTO v_evaluacion;
INSERT INTO cirugia (solicitud_cirugia_id,evaluacion_solicitud_cirugia_id,
quirofano_id,fecha_programada,fecha_inicio,fecha_fin,estado,procedimiento_realizado)
VALUES (v_solicitud,v_evaluacion,v_quirofano,'2026-09-17 09:00:00-06',
'2026-09-17 09:00:00-06','2026-09-17 10:00:00-06','Finalizada',
'Intervencion ficticia concluida')
RETURNING cirugia_id INTO v_cirugia;
INSERT INTO participante_cirugia (cirugia_id,persona_id,rol)
VALUES (v_cirugia,v_m2,'Cirujano'),(v_cirugia,v_m1,'Anestesiologo'),
(v_cirugia,v_enfermero_persona,'Enfermero');
INSERT INTO recurso (nombre,categoria,tipo,material,costo_unitario)
VALUES ('Insumo esteril ficticio','Insumo','Quirurgico','Material de prueba',100)
RETURNING recurso_id INTO v_recurso;
INSERT INTO solicitud_recurso (solicitud_cirugia_id,recurso_id,cantidad)
VALUES (v_solicitud,v_recurso,1);
INSERT INTO uso_recurso_cirugia

```

```

(cirugia_id,recurso_id,cantidad,costo_unitario_aplicado,fecha_uso)
VALUES (v_cirugia,v_recurso,1,100,'2026-09-17 09:20:00-06');

INSERT INTO consentimiento_quirurgico
(cirugia_id,medico_firmante_id,firmante_persona_id,calidad_firmante,
procedimiento,objetivo,caracteristicas,riesgos,fecha_obtencion,
firma_medico_confirmada,firma_paciente_o_representante_confirmada)
VALUES (v_cirugia,v_m2,v_p3,'Paciente','Intervencion simulada',
'Prueba academica','Procedimiento sin pacientes reales',
'Riesgos ficticios explicados','2026-09-17 08:00:00-06',TRUE,TRUE);

INSERT INTO chequeo_preanestesico
(cirugia_id,medico_evaluador_id,anestesista_id,
clasificacion_asa,plan_anestesia,fecha_evaluacion,firma_evaluador_confirmada)
VALUES (v_cirugia,v_m2,v_m1,'I','Plan simulado',
'2026-09-17 08:10:00-06',TRUE);

INSERT INTO etapa_cirugia
(cirugia_id,enfermero_responsable_id,etapa,fecha_inicio,fecha_fin,
fecha_envio_secretaria)
VALUES (v_cirugia,v_empleado,'Preoperatorio','2026-09-17 08:00:00-06',
'2026-09-17 09:00:00-06','2026-09-17 11:00:00-06')

RETURNING etapa_cirugia_id INTO v_etapa;

INSERT INTO item_control_quirurgico (etapa,grupo,descripcion,tipo_resultado)
VALUES ('Preoperatorio','Entrada','Identidad confirmada','ExitoFalla')

RETURNING item_control_quirurgico_id INTO v_item;

INSERT INTO resultado_control_quirurgico
(etapa_cirugia_id,item_control_quirurgico_id,etapa,resultado,
fecha_registro,registrado_por_enfermero_id)
VALUES (v_etapa,v_item,'Preoperatorio','Exito',
'2026-09-17 08:20:00-06',v_empleado);

INSERT INTO ingreso (episodio_atencion_id,hospital_id,unidad_medica_id,
servicio_medico_id,medico_encargado_id,espacio_hospitalario_id,
fecha_ingreso,motivo_ingreso,diagnostico_presuntivo)
VALUES (v_ep3,v_hosp,v_cir,v_scir,v_m2,v_quirofano,
'2026-09-17 08:45:00-06','Cirugia programada',
'Diagnostico quirurgico simulado')

RETURNING ingreso_id INTO v_i3;

INSERT INTO egreso (ingreso_id,medico_asignado_id,fecha_egreso,motivo_egreso,codigo_egreso)
VALUES (v_i3,v_m2,'2026-09-17 12:00:00-06','Alta posoperatoria','Vivo');

UPDATE ingreso SET estado='Cerrado' WHERE ingreso_id=v_i3;

INSERT INTO factura (numero_factura,paciente_id,hospital_id,ingreso_id,descripcion)
VALUES ('FAC-DEMO-003',v_p3,v_hosp,v_i3,'Cirugia de demostracion')

```

```

RETURNING factura_id INTO v_f;

INSERT INTO factura_detalle (factura_id,concepto,cantidad,precio_unitario)

VALUES (v_f,'Procedimiento quirurgico',1,2500),

(v_f,'Insumo esteril',1,100);

INSERT INTO pago (factura_id,unidad_recepcion_id,fecha_pago,monto,metodo)

VALUES (v_f,v_cir,'2026-09-17 12:15:00-06',2600,'Tarjeta');

UPDATE factura SET estado='Pagada' WHERE factura_id=v_f;

-- Hospitalizacion: ingreso, ocupacion de cama y costo diario.

INSERT INTO episodio_atencion (paciente_id,hospital_id,fecha_apertura,motivo_apertura)

VALUES (v_p4,v_hosp,'2026-09-18 10:00:00-06','Observacion hospitalaria')

RETURNING episodio_atencion_id INTO v_ep4;

INSERT INTO ingreso (episodio_atencion_id,hospital_id,unidad_medica_id,

servicio_medico_id,medico_encargado_id,espacio_hospitalario_id,

fecha_ingreso,motivo_ingreso,diagnostico_presuntivo)

VALUES (v_ep4,v_hosp,v_hospit,v_shosp,v_m1,v_cama,

'2026-09-18 10:00:00-06','Observacion simulada',

'Necesidad de vigilancia clinica ficticia')

RETURNING ingreso_id INTO v_i4;

INSERT INTO hospitalizacion

(ingreso_id,medico_encargado_id,fecha_alta_prevista,fecha_alta_real,costo_diario_aplicado)

VALUES (v_i4,v_m1,'2026-09-19 10:00:00-06','2026-09-19 10:00:00-06',350)

RETURNING hospitalizacion_id INTO v_estadia;

INSERT INTO asignacion_cama (hospitalizacion_id,cama_id,fecha_inicio,fecha_fin)

VALUES (v_estadia,v_cama,'2026-09-18 10:00:00-06','2026-09-19 10:00:00-06');

INSERT INTO hospitalizacion_servicio

(hospitalizacion_id,servicio_medico_id,medico_id,fecha_servicio,descripcion,costo_aplicado)

VALUES (v_estadia,v_shosp,v_m1,'2026-09-18 11:00:00-06',

'Atencion medica de demostracion',150);

INSERT INTO hospitalizacion_insumo

(hospitalizacion_id,recurso_id,fecha_uso,cantidad,costo_unitario_aplicado)

VALUES (v_estadia,v_recurso,'2026-09-18 12:00:00-06',1,100);

INSERT INTO egreso (ingreso_id,medico_asignado_id,fecha_egreso,motivo_egreso,codigo_egreso)

VALUES (v_i4,v_m1,'2026-09-19 10:00:00-06','Fin de observacion','Vivo');

UPDATE ingreso SET estado='Cerrado' WHERE ingreso_id=v_i4;

UPDATE episodio_atencion

SET estado='Cerrado',fecha_cierre='2026-09-19 10:00:00-06'

WHERE episodio_atencion_id=v_ep4;

INSERT INTO factura (numero_factura,paciente_id,hospital_id,ingreso_id,descripcion)

VALUES ('FAC-DEMO-004',v_p4,v_hosp,v_i4,'Hospitalizacion de demostracion')

```

```

RETURNING factura_id INTO v_f;

INSERT INTO factura_detalle (factura_id,concepto,cantidad,precio_unitario)

VALUES (v_f,'Un dia de hospitalizacion',1,350),

(v_f,'Servicio medico',1,150),

(v_f,'Insumo utilizado',1,100);

INSERT INTO pago (factura_id,unidad_recepcion_id,fecha_pago,monto,metodo)

VALUES (v_f,v_hospit,'2026-09-19 10:15:00-06',600,'Transferencia');

UPDATE factura SET estado='Pagada' WHERE factura_id=v_f;

INSERT INTO evaluacion_calidad

(paciente_id,hospital_id,ingreso_id,fecha_evaluacion,observaciones)

VALUES (v_p4,v_hosp,v_i4,'2026-09-19 11:00:00-06',

'Evaluacion ficticia de demostracion')

RETURNING evaluacion_calidad_id INTO v_calidad;

INSERT INTO evaluacion_objetivo

(evaluacion_calidad_id,hospital_objetivo_id,nota,comentario)

VALUES (v_calidad,v_hosp,5,'Atencion simulada satisfactoria');

INSERT INTO evaluacion_objetivo

(evaluacion_calidad_id,persona_objetivo_id,nota,comentario)

VALUES (v_calidad,v_m1,4,'Calificacion ficticia del medico');

END

$$;

COMMIT;

/* Comprobacion: cuatro pacientes, cuatro unidades y cuatro facturas. */

SELECT (SELECT COUNT(*) FROM hospital.paciente) AS pacientes,

(SELECT COUNT(*) FROM hospital.unidad_medica) AS unidades,

(SELECT COUNT(*) FROM hospital.factura) AS facturas;

```

roles_privilegios.sql

```
BEGIN;

DO $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'hospital_lectura') THEN
        CREATE ROLE hospital_lectura NOLOGIN;
    END IF;

    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'hospital_admin') THEN
        CREATE ROLE hospital_admin NOLOGIN;
    END IF;
END
$$;

GRANT CONNECT ON DATABASE hospitales_occidente TO hospital_lectura, hospital_admin;
GRANT USAGE ON SCHEMA hospital TO hospital_lectura;
GRANT USAGE, CREATE ON SCHEMA hospital TO hospital_admin;

-- El rol administrativo pasa a ser propietario del esquema y sus tablas.
-- La propiedad permite ALTER y DROP, que GRANT ALL por si solo no concede.
DO $$
DECLARE
    objeto RECORD;
BEGIN
    FOR objeto IN
        SELECT c.relname, c.relkind
        FROM pg_class c
        JOIN pg_namespace n ON n.oid = c.relnamespace
        WHERE n.nspname = 'hospital'
        AND c.relkind IN ('r', 'p', 'v', 'm')
        ORDER BY c.relkind, c.relname
    LOOP
        CASE objeto.relkind
            WHEN 'r', 'p' THEN
                EXECUTE format('ALTER TABLE hospital.%I OWNER TO hospital_admin', objeto.relname);
            WHEN 'v' THEN
                EXECUTE format('ALTER VIEW hospital.%I OWNER TO hospital_admin', objeto.relname);
            WHEN 'm' THEN
                EXECUTE format('ALTER MATERIALIZED VIEW hospital.%I OWNER TO hospital_admin', objeto.relname);
        END CASE;
    END LOOP;
END;
```

```

        END LOOP;

END

$$;

ALTER SCHEMA hospital OWNER TO hospital_admin;


-- Auditoría: consultas SELECT, sin INSERT, UPDATE, DELETE ni DDL.
GRANT SELECT ON ALL TABLES IN SCHEMA hospital TO hospital_lectura;


-- Administración de todos los objetos existentes en el esquema.
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA hospital TO hospital_admin;
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA hospital TO hospital_admin;
GRANT ALL PRIVILEGES ON ALL FUNCTIONS IN SCHEMA hospital TO hospital_admin;


-- Los futuros objetos creados por postgres heredarán esos permisos.
ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA hospital
    GRANT SELECT ON TABLES TO hospital_lectura;
ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA hospital
    GRANT ALL PRIVILEGES ON TABLES TO hospital_admin;
ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA hospital
    GRANT ALL PRIVILEGES ON SEQUENCES TO hospital_admin;
ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA hospital
    GRANT ALL PRIVILEGES ON FUNCTIONS TO hospital_admin;
ALTER DEFAULT PRIVILEGES FOR ROLE hospital_admin IN SCHEMA hospital
    GRANT SELECT ON TABLES TO hospital_lectura;


COMMIT;


/*

Para crear usuarios concretos mas adelante, dentro de psql:

CREATE ROLE auditor_hospital LOGIN PASSWORD 'contraseña elegida por ti';
GRANT hospital_lectura TO auditor_hospital;

CREATE ROLE gestor_hospital LOGIN PASSWORD 'otra contraseña elegida por ti';
GRANT hospital_admin TO gestor_hospital;


Nunca guarde contraseñas reales en este archivo ni en Git.


Verificación de privilegios:

SELECT
    has_table_privilege('hospital_lectura','hospital.paciente','SELECT') AS lectura_select,
    has_table_privilege('hospital_lectura','hospital.paciente','INSERT') AS lectura_insert,
```

```
has_table_privilege('hospital_admin','hospital.paciente','INSERT') AS admin_insert;
```

Resultado esperado: true, false, true.

El rol hospital_admin es propietario de las tablas y del esquema.

Los miembros a los que se otorgue este rol podran administrar su contenido y estructura sin concederles privilegios de superusuario.

*/

5. Comprobación y resultados

La carga ficticia produjo cuatro pacientes, cuatro unidades y cuatro facturas. La verificación V01-V14 confirmó 64 tablas, 159 restricciones CHECK, 124 relaciones foráneas y permisos diferenciados. La factura de emergencias registra Q500.00, con Q250.00 pagados y Q250.00 pendientes. Se generó un respaldo personalizado mediante `pg_dump -Fc`.

6. Conclusiones

El diseño vincula las cuatro unidades médicas con las fichas clínicas, los servicios y sus cobros. Las claves y restricciones protegen los dominios y las relaciones; los roles separan consulta de administración. Las reglas temporales y de sumas entre varias filas deben verificarse adicionalmente en los procesos de registro o con lógica transaccional.